

potentially useful for the development of the project, the *SWRL API* implementation is still included in the project.

2.3 Load data in a table format (.csv) to an ontology file (.owl) using *CSVtoOntologyExec*

When executed, the class *CSVtoOntologyExec* will read the content in the specified data base in a .csv file and load the information into an ontology file (.owl). An important consideration is that the .owl files should be written in *RDF/XML* syntax (choose that option when using *Save as* in *Protégé*). In what concerns to *myCBR* retrieval, this class may only be executed when the case base information and similarity values contained in the .prj file should be changed or updated to perform CBR queries using new data. The path for working file folder should have been set in the *AppConfiguration* class, together with the corresponding files denominations. In this folder, it is recommended to locate a clean version of the ontology that is wanted to be used (without instances or individual or property assertions) and another copy (of course with a different name) where the data base will be stored. So, when updating the data base the procedure should be as follows:

- Delete the data base ontology file.
- Make a copy of the clean ontology file and change the name as desired . Of course, the same name should be specified in the *AppConfiguration* class (*ont_file_name*).
- Execute *CSVtoOntologyExec*, which will read the clean ontology file (specified in *AppConfiguration* as *base_ont_file_name*) and the data base of the .csv file (specified in *AppConfiguration* as *csv*) to merge the information in the ontology data base file.

The translation of the information stored in the table to ontological entities is stated with the appropriate using of the methods of the class *CSVtoOntology* in the package *OntologyTools*. See the *javadoc* of the project for details. By this way, the code is flexible to adapt to the ontological meaning of the content in the different columns and cells forming the table tabular data base.

In this particular case, the executable code in the *CSVtoOntologyExec* class is configured for the Predictive Maintenance data base and the *OPMAD* ontology. Nevertheless, the class could be modified if needed to suit to another different case or to adapt to a restructuring of the current data base and ontology.

2.4 Load data from an ontology file (.owl) to a table format .csv file using *OntologytoCSVExec*

The class *OntologytoCSVExec* is an executable class allowing to extract data from an ontology file and rewrite it in an organized table format into a .csv file. The class *OntologytoTabular* is used to get the data and organize it in a structured tabular *List* object. In the mentioned class, a *Jena API* implementation is used to be able to execute a series of *SPARQL* queries on the working ontology and get a tabular data as a result. The advantages of this type of queries for this application are their execution time (fast execution as they do not need reasoning) and the retrieval results obtained on the shape of organized tables. Now, one important issue should be underlined: the *SPARQL* queries have to be adapted to the particular ontology of the user, as well as the organization criteria for the data when stored as a .csv table. Even if it is possible to use one single query for the data extraction, it is recommended to divide the query in various of them when the size of the database is remarkable, as it would be much faster to execute. The list of queries is automatically built by a method in the *AppConfiguration* file. It is required to specify the static *String* parameter *queryHead* in the mentioned class to set the first part of the *SPARQL* queries, that will be common to all the queries in the list. This part of the query aims to define the variables that are going to be retrieved and to initialize the selection block. The closing of the query is also common to all the queries in the list, so it can be specified with the parameter *queryEnd*. The body commands of the queries, which will use the relations existing in the ontology to extract the data, are listed in the parameter *queryBodyList* in the *AppConfiguration* file. Then, the method *queryList* will build the strings with the full queries adding one head and one closing to each one of the body commands blocks. A reference to the *SPARQL* syntax can be found at [11]. The general procedure to determine the query structure is to choose the variables defining the cases, establish which entities in the ontology contain that information and look for properties or ontological relations that build links between the different pieces of data.

The output data of a *SPARQL* query is organized in columns, one for each requested variable. Cells in the table can store one single data value. If a variable value in one of the columns matches more than one value of other variable, then there will be one row in the query results table for each combination of values. That comes to say, using a simplified example, that having a table with only two columns in which each one of the n values of the first variable matches two values of the second variable, then the table will have $2n$ rows, where the values of the first variable will appear twice in the column. The same logic extends to several variables with multiple values, that would imply to add as many rows as necessary to include all the combinations of values. An example for the application case of Predictive Maintenance Decision-Support System is shown in Figure 1, where it is observed that there are as many rows concerning the reference index '1' as values of the field *Input type* existing for such case. Note that the indices are ordered using the ontological *Protégé* criteria instead of mathematical order.

Reference	Publication_Year	Task	Case_study	Case_study_type	Input_for_the_model	Input_type
"1"	2019	Health modelling	Simulated_jet-engines_data	Rotary_machines	Time_series	Bypass_ratio
"1"	2019	Health modelling	Simulated_jet-engines_data	Rotary_machines	Time_series	Temperature
"1"	2019	Health modelling	Simulated_jet-engines_data	Rotary_machines	Time_series	Fluid_Pressure
"1"	2019	Health modelling	Simulated_jet-engines_data	Rotary_machines	Time_series	Spinning_speed
"10"	2016	One_step_future_state_forecast	Proton_exchange_membrane_fuel_cell	Energy_cells_and_batteries	Time_series	Voltage
"100"	2013	Multiple_steps_future_state_forecast	Lithium-ion_battery	Energy_cells_and_batteries	Time_series	Impedance
"100"	2013	Multiple_steps_future_state_forecast	Lithium-ion_battery	Energy_cells_and_batteries	Time_series	Temperature
"100"	2013	Multiple_steps_future_state_forecast	Lithium-ion_battery	Energy_cells_and_batteries	Time_series	Voltage
"100"	2013	Multiple_steps_future_state_forecast	Lithium-ion_battery	Energy_cells_and_batteries	Time_series	Current

Figure 1: Example of SPARQL query results table for the application Predictive Maintenance Decision-Support System.

Additionally, a complete *SPARQL* query for the previously mentioned application case is included in Figure 2 as example to be modified by the user. There is an important detail that must be clarified in what concerns to the variable designations in the *SPARQL* query: as the characters blank space, '-' and '/' are not allowed, they are substituted respectively by '_', double '_' and triple '_'. When rewriting data in a tabular shape using the class *OntologytoTabular*, the headers of the table containing the variables designations are automatically recovered following the inverse transformation rule.

To optimize the execution time of the queries, they can be divided. Individual queries can be executed to obtain partial data tables in which the reference indices column will be extracted together with one or more of the other columns. It is necessary that all the partial queries get the reference indices column as the first column of their results table, so as later all the partial tables can be merged in one. Hence, in most of the cases the columns can be extracted individually, but sometimes it could be required that some columns are extracted through the same query. When two or more columns that store multiple elements in their cells have an order dependency between them, they must be associated to the same query. The order dependency means that the elements listed in the cells of one of the columns are linked to another corresponding element among the ones listed in the cell of another columns. So, the elements in both columns must be listed in the same order to preserve the relation between the elements of data.

Once defined the queries, the results coming from their execution will be rewritten to tables where each row should match only one case, merging the rows coming from the *SPARQL* result to gather in the correspondent cells the set of values for those fields that are multiple-valued. A reference column is chosen (first column) with integer reference indices so as the content of the cells of all the rows with the same index is merged. The general criteria to do so is that no value is repeated in the multiple values cells. However, a set of variables names can be listed using the parameter *Repetition_allowed* for those columns in which repeated values in the same cell should be allowed. As blank spaces are forbidden in the *SPARQL* syntax, when the text values are recovered they are rewritten substituting '_' characters by blank spaces, unless the user specifies not to do so via the parameter *Not_Spaced*. At the end, all the tables will be put together according to the reference indices column, so as the data coming from the list of queries is gathered into one single data table.

Then, here are the features that must be set up in order to define the data transference from the ontology to a *.csv* file:

- *SPARQL* queries list that will determine what information is requested to the ontology and how it is organized. To be set up in the *AppConfiguration* file.
- Parameter list *Not_Spaced* to be specified in the class *AppConfiguration*. In this list it must be included the designation tags of those variables or fields in the table where blank spaces are not desired or expected. For those columns of the table, the text value will be written as it is in origin in the query results, without replacing characters ' ' by blank spaces.
- Parameter list *Repetition_allowed* to be specified in the class *AppConfiguration*. In this list the user must include, among those variables (columns) that could contain multiple elements in the same cell, in which of them it is allowed that elements are repeated more than once in the same cell.

For the example below, concerning the Predictive Maintenance Decision-Support System application, it is illustrated the structure of the *SPARQL* query (Figure 2), the results for a case with multiple values (Figure 3) and the final format in the *.csv* table. So, when the results are retrieved from the *SPARQL* query, more than one row may be obtained for each case to get all the values combinations of all the fields that are multiple-valued. In the Figure 3 it is observed how the case with reference index '8' has multiple values for the fields *Input type* and *Models*, while the description tags of the field *Model Type* are matched to particular values of *Models*. In this example, a single query has been used in order to clarify the explanation, but in the real application case the query was divided in parts. Indeed, a query was used for each of the columns in the table except the pair *Model Type* and *Models* and the pair *Performance indicator* and *Performance*, which were respectively included in the same query. The reason for that is that they must keep an order relation for the elements that are listed in the cells. The *Model Type* values must be stored in the same order that the corresponding elements in the column *Models* that they are describing. Same condition is needed for the elements listed in *Performance indicator* and the their value in the column *Performance*. The column *Model Type* is declared in the list *Repetition_allowed*, so as more than one of the *Models* concerning one case could be described with the same type if they belong to the same family of models.

```

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX def: <http://www.semanticweb.org/j.montero-jimenez/ontologies/2021/2/OPNAD#>
PREFIX obo: <http://purl.obolibrary.org/obo/>
PREFIX cco: <http://www.ontologyrepository.com/CommonCoreOntologies/>

SELECT ?Reference ?Publication_Year ?Task ?Case_study ?Case_study_type ?Input_for_the_model ?Input_type ?Model_Type
?Models ?Online_Off__line ?Performance_indicator ?Performance ?Complementary_notes ?Study_title ?Publication_identifier
WHERE {

    ?a rdf:type def:Predictive_maintenance_system_module .
    ?a obo:RO_0010002 ?d .
    ?b obo:RO_0010002 ?d .
    ?b rdf:type def:Predictive_Maintenance_Article .
    ?Models rdfs:subClassOf def:Predictive_maintenance_model .
    ?d rdf:type ?Models .
    ?e rdf:type def:Predictive_maintenance_case .
    ?e cco:designates ?a .
    ?e def:has_text_value ?Reference .
    ?b def:has_publication_year ?Publication_Year .
    ?Task rdfs:subClassOf def:Predictive_maintenance_module_function .
    ?h rdf:type ?Task .
    ?a def:has_predictive_maintenance_function ?h .
    ?a obo:BF0_0000051 ?j .
    ?j rdf:type ?Case_study .
    ?Case_study rdfs:subClassOf def:Maintainable_item .
    ?Case_study_type rdf:type def:item_type .
    ?b obo:RO_0010002 ?Case_study_type .
    ?Input_for_the_model rdfs:subClassOf def:maintainable_item_record .
    ?n rdf:type ?Input_for_the_model .
    ?a obo:BF0_0000051 ?n .
    ?Input_type rdf:type def:Data_variable .
    ?n obo:RO_0010002 ?Input_type .
    ?Model_Type rdf:type def:Model_type .
    ?Model_Type cco:describes ?d .
    ?a def:has_synchronization ?Online_Off__line .
    ?Online_Off__line rdf:type def:Module_synchronization .
    ?b def:has_title ?r .
    ?r def:has_text_value ?Study_title .
    ?b def:has_identifier ?s .
    ?s def:has_text_value ?Publication_identifier .
    ?Performance_indicator rdfs:subClassOf def:Performance_value .
    ?t rdf:type ?Performance_indicator .
    ?t cco:describes ?a .
    ?t def:has_text_value ?Performance .
    ?u rdf:type def:Complementary_notes .
    ?u cco:describes ?b .
    ?u def:has_text_value ?Complementary_notes .

}ORDER BY (?Reference)

```

Figure 2: Example of complete SPARQL for the application Predictive Maintenance Decision-Support System.

Reference	Publication_Year	Task	Case_study	Case_study_type	Input_for_the_model	Input_type	Model_Type	Models
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Plastic_strains	Data-driven	Bayes_model
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Mechanical_Pressure	Data-driven	Bayes_model
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Life_cycles	Data-driven	Bayes_model
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Mechanical_stresses	Data-driven	Bayes_model
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Elastic_strains	Data-driven	Bayes_model
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Plastic_strains	Physics-based	Particle_Filter
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Mechanical_Pressure	Physics-based	Particle_Filter
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Life_cycles	Physics-based	Particle_Filter
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Mechanical_stresses	Physics-based	Particle_Filter
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Elastic_strains	Physics-based	Particle_Filter
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Plastic_strains	Physics-based	Physics-based_model_for_track_settlement
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Mechanical_Pressure	Physics-based	Physics-based_model_for_track_settlement
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Life_cycles	Physics-based	Physics-based_model_for_track_settlement
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Mechanical_stresses	Physics-based	Physics-based_model_for_track_settlement
"8"	2019	Remaining useful life estimation	Railway_track_geometry	Structures	Time_series	Elastic_strains	Physics-based	Physics-based_model_for_track_settlement

(a)

Online_Off_line	Performance_indicator	Performance	Complementary_notes	Study_title	Publication_identifier
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa "A_knowledge-based_prognostics_framework_"	"doi.org/10.1016/j.res.2018.07.004"	
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa "A_knowledge-based_prognostics_framework_"	"doi.org/10.1016/j.res.2018.07.004"	
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa "A_knowledge-based_prognostics_framework_"	"doi.org/10.1016/j.res.2018.07.004"	
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa "A_knowledge-based_prognostics_framework_"	"doi.org/10.1016/j.res.2018.07.004"	
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa "A_knowledge-based_prognostics_framework_"	"doi.org/10.1016/j.res.2018.07.004"	
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa "A_knowledge-based_prognostics_framework_"	"doi.org/10.1016/j.res.2018.07.004"	
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa "A_knowledge-based_prognostics_framework_"	"doi.org/10.1016/j.res.2018.07.004"	
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa "A_knowledge-based_prognostics_framework_"	"doi.org/10.1016/j.res.2018.07.004"	
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa "A_knowledge-based_prognostics_framework_"	"doi.org/10.1016/j.res.2018.07.004"	
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa "A_knowledge-based_prognostics_framework_"	"doi.org/10.1016/j.res.2018.07.004"	
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa "A_knowledge-based_prognostics_framework_"	"doi.org/10.1016/j.res.2018.07.004"	
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa "A_knowledge-based_prognostics_framework_"	"doi.org/10.1016/j.res.2018.07.004"	
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa "A_knowledge-based_prognostics_framework_"	"doi.org/10.1016/j.res.2018.07.004"	
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa "A_knowledge-based_prognostics_framework_"	"doi.org/10.1016/j.res.2018.07.004"	
Online	Mean_absolute_percentage_error	"(<5%)"	"No_info_about_operationa "A_knowledge-based_prognostics_framework_"	"doi.org/10.1016/j.res.2018.07.004"	

(b)

Figure 3: SPARQL query results table for one particular case for the application Predictive Maintenance Decision-Support System.

Reference	Publication_Year	Task	Case_study	Case_study_type	Input_for_the_model	Input_type	Data_Pre-proc	Model_Appro	Model_Type	Models
1	2019	Health assess	Simulated jet-engines data	Rotary machine	Time series	Temperature, yes		Single model	Data-driven	Logistic regression
2	2019	Remaining useful life estimation	Simulated jet-engines data	Rotary machine	Time series	Health index, yes		Single model	Data-driven	OS-ELM (Online sequential e
3	2019	Remaining useful life estimation	Simulated jet-engines data	Rotary machine	Time series	Health index, yes		Multi model	Data-driven	KFOS-ELM (Kalma filter-base
4	2019	Remaining useful life estimation	Simulated jet-engines data	Rotary machine	Time series	Health index, yes		Multi model	Data-driven	EOS-ELM (Ensemble of OS-ELI
5	2019	Remaining useful life estimation	Simulated jet-engines data	Rotary machine	Time series	Health index, yes		Multi model	Data-driven	AEKFOS-ELM (adaptive-veig
6	2019	Remaining useful life estimation	Simulated jet-engines data	Rotary machine	Time series	Health index, yes		Multi model	Data-driven	Physics-based model for trac
7	2019	Health assess	Railway track geometry	Structures	Time series	Mechanical st No		Multi model	Physics-based, Data-driven	Physics-based model for trac
8	2019	Remaining useful life estimation	Railway track geometry	Structures	Time series	Mechanical st No		Multi model	Physics-based, Data-driven, Physics-based	Physics-based model for trac

(a)

Online/Off-line	Performance indicator	Performance	Complementary notes	Study title	Publication identifier
Off-line	Mean absolute percentage error	(<0.05)	1 operational mode, 10t Aircraft engine	doi.org/10.1016/j.ast.2018.09.044	
Off-line	Mean absolute percentage error	(<0.05)	1 operational mode, 10t Aircraft engine	doi.org/10.1016/j.ast.2018.09.044	
Off-line	Score function (the smaller the better), Mean accuracy, Error range	(58.96), (93.26), ([21,33])	1 operational mode, 10t Aircraft engine	doi.org/10.1016/j.ast.2018.09.044	
Off-line	Score function (the smaller the better), Mean accuracy, Error range	(35.78), (95.78), ([15,22])	1 operational mode, 10t Aircraft engine	doi.org/10.1016/j.ast.2018.09.044	
Off-line	Score function (the smaller the better), Mean accuracy, Error range	(34.56), (96.54), ([13,19])	1 operational mode, 10t Aircraft engine	doi.org/10.1016/j.ast.2018.09.044	
Off-line	Score function (the smaller the better), Mean accuracy, Error range	(33.13), (96.76), ([11,18])	1 operational mode, 10t Aircraft engine	doi.org/10.1016/j.ast.2018.09.044	
Off-line	N/A	N/A	No info about operator A knowledge-b	doi.org/10.1016/j.res.2018.07.004	
Online	Mean absolute percentage error	(<5%)	No info about operator A knowledge-b	doi.org/10.1016/j.res.2018.07.004	

(b)

Figure 4: Final results table as it is loaded in the .csv file for the application Predictive Maintenance Decision-Support System.

2.5 Preparing project case base and similarity values for *myCBR* with *myCBRSetting*

The class *myCBRSetting* may be executed when the data base in the *.csv* (file name specified as *csv* in the class *AppConfiguration*) file or the ontology *.owl* file have been changed (normally their modifications are coordinated). In order to perform *myCBR* queries on a new case base, or a case base that has been updated, the information must be written in the *.prj* (file name specified as *projectName* in the class *AppConfiguration*) file, which is the one that contains the project data for *myCBR*. Moreover, the class will run an *OWL* reasoner (more precisely the *HermiT* reasoner, see Section 3) on the ontology that will check its consistency and infer relations for the knowledge contained in the ontology. In consequence, the execution of the class can take a bit of minutes, depending on the ontology size. The main reason to use the reasoner is to be able to perform queries on the ontology to extract information that can be used to calculate ontological similarity values. Moreover, the *HermiT* reasoner allows to make queries about relations that are not initially explicit in the ontology but inferred during the reasoning process. Obviously, if the ontology is not consistent the execution will stop, so to work with an ontological data base it must be consistent. Furthermore, even if the data base can be loaded to the project file either from a *.csv* data file or from an *.owl* ontology, the ontology file is always necessary to obtain the similarity values. The class *myCBRSetting* may be modified to adapt the code to a new application other than the Predictive Maintenance Decision-Support System. This is because the ontology relations and the method for semantic similarity calculation depend obviously on the application.

When executing, at first, the class uses an instance of *CBREngine* to load the current *.prj* file, delete all the existing instances in the case base and introduce the new ones, with their corresponding attribute values, by importing the chosen *.csv* file or the *.owl* ontology file. If the *.owl* database is chosen, the procedure to extract the data is exactly the same as the one followed when executing *OntologytoCSVExec*, see the Subsection 2.4 for more details. The class *OntologytoTabular* will get the data from the ontology following the structure of the *SPARQL* queries list. Then, once the data has been tabulated, it is loaded directly into the *myCBR* project file instead of creating a *.csv* file. So, using the direct importing from the *.owl* file has the same result that creating the *.csv* file from the ontology and loading it with the default *myCBR* importer.

After that, the similarity functions are established with the appropriate values. In particular for the Predictive Maintenance Decision-Support System application, for the field *Task* associated to each one of the cases, the similarity values are calculated using an ontological method. The querying of the ontology uses the classes *DLQueryEngineIRI* and *DLQueryParserIRI* of the package *OntologyTools*. See the *javadoc* of the project for more details.

Case variable	Variable type (<i>myCBR</i>)	Values
<i>Task</i>	Symbol	Fault feature extraction, Fault detection, Fault identification, Health modelling, Health assessment, Remaining useful life estimation, One step future state forecast, Multiple steps future state forecast.
<i>Case study type</i>	Symbol	Rotary machines, Reciprocating machines, Electrical components, Structures, Energy cells and batteries, Production lines, Others.
<i>Case study</i>	String	The <i>myCBR</i> Levenshtein function is used. Similarity is calculated with the quotient $\frac{\text{number of characters reference String} - \text{Levenshtein distance}}{\text{number of characters reference String}}$
<i>Input type</i>	Symbol	A list of variables must be provided separated by ' , ' and where all the words should begin with capital letters as it is established in the case base. An additional Levenshtein method in <i>Java</i> allows to support misspelling up to 3 erroneous characters.
<i>Online/Off-line</i>	Symbol	Online, Off-line, Both, Unknown synchronization.
<i>Input for the model</i>	Symbol	Signals, Structured text-based, Text based maintenance/ operations logs, Time series.
<i>Publication Year</i>	Integer	This field is not provided by the user, the application will used the current date automatically. The most recent cases in the case base will be prioritized over the older ones.

Table 1: Case variables for querying and retrieval

2.6 Query and retrieval using the *GUI*'s

Once executed *CSVtoOntologyExec* to load the data base into the ontology and *myCBRSetting* to configure the *.prj* file for *myCBR*, only the executable *GUI*'s are required to query and retrieve until the case base is wanted to be modified. The tool *myCBR* uses the case base, the attributes and the similarity functions specified in the *.prj* file to search the most suitable case for the given query. To visualize or to modify manually the project file the *myCBR Workbench* application may be used.

When executing any one of the *GUI*'s, the class *Recommender* is used. Most of the similarity functions are defined during the execution of the class *myCBRSetting*, but there is one particular field in the Predictive Maintenance Decision-Support System which similarity values are defined by comparing the current query to all the cases in the data base individually, and that is *Input type*. An analog method is used, which is analog to the one applied to establish the similarity values between the different Predictive Maintenance functions (field *Task*).

Two executable *Graphical User Interfaces* are provided for querying: *GUI2* and *GUI3*.

2.6.1 *GUI2*

The *GUI2* allows the user to perform one query at a time by specifying the following parameters:

- Values of the case fields for the retrieval. Some of them must be typed (*Case Study* and *Input type*) and for the rest of the fields the values are selected from the ones available in a drop down menu.
- Value of the weights assigned to each field.
- Type of amalgamation function that is used to get the global similarity value of each case.
- Number of cases to be retrieved. The resulting list of cases (ordered from higher to lower similarity value) will be shown in the screen after submitting the query.

Input variables		Variable weights	
Task :	Feature extraction		1.0
Case study type :	Rotary machines		1.0
Case study :	Simulated jet-engines data		1.0
Input type :	Temperature, Fluid Pressure, Spinning speed, Baypass		1.0
Online/Off-line :	Online		1.0
Input for the model :	Signals		1.0

Additional inputs

Number of cases to retrieve: 3

Amalgamation function to use: euclidean

User dialog:

Welcome to the myCBR Graphical User Interface !

* Input Variables : variables used in the query to retrieve and calculate similarities.
 - Task, Publication year, Case Study Type, Online/Off-line, Input for the model, Input type : Drop down list
 - Case Study : Free text

* Additional inputs : inputs to complement the retrieval method.
 - Number of cases to retrieve : Integer number.
 - Amalgamation function to use : Drop down list

SUBMIT QUERY

Figure 5: Window of the *GUI2*

If one of the fields is left blank, then its weight in the global similarity value is automatically set to 0. The value of the field *Case study* is expected to be equal to one already existing in the data base, otherwise the similarity value will be 0 for all the cases.

The field *Input type* contains a list of variables (most of them physical variables) that are considered in the Predictive Maintenance model of each case. This list must be typed with the terms separated by ', ' and all the words starting by capital letters, as they appear in the case base. Nevertheless, both fields can manage with possible misspelling errors using the Levenshtein distance method. In particular, the *Case study* field is defined as string type in *myCBR*, and it uses the default Levenshtein comparison function. But, for the *Input type* field, the Levenshtein method is not available in *myCBR* as it is declared as symbolic value. So, an additional method (class *LevenshteinDistanceDP*) to allow up to a distance of three erroneous characters in the spelling is implemented in the similarity function definition (*Recommender*). See the *javadoc* for more details.

2.6.2 GUI3

Using this *GUI*, the user is able to execute a list of consecutive queries provided in an input file in *.csv* format and to save their results in separate files (one for each query in the list). It is necessary to prepare an input file with the appropriate structure (see Figure 6). An example of input file is also provided in the actual project folder of the application. For the fields that are stated as symbolic in *myCBR* and for the string variable *Case study*, the user must type in the input file a value which is included among the possible values for each field (see Table 1), otherwise the similarity will be just 0. Symbolic fields, with the exception of *Input type*, do not support misspelling. The field *Input type* contains a list of variables separated by ', ' where the words should begin by capital letters (as they are in the data base). As soon as one of the variables of the list exist in one of the cases in the data base the similarity value will not be 0 for that case.

Task	w1	Case study ty/w2	Case study w3	Online/Offline w4	Input for the model w5	Input type w6	Number of ca Amalgamation function
Fault detectio	1	Rotary machi	1 Simulated jet-	1 Online	1 Time series	1 Temperature	1 20 euclidean
Feature extra	1	Rotary machi	1 Simulated jet-	1 Offline	1 Time series	1 Temperature	1 1 euclidean
Fault detectio	1	Rotary machi	1 Simulated jet-	1 Offline	1 Time series	1 Temperature	1 5 euclidean

Figure 6: Example *.csv* input file for the *GUI3*

After having prepared the input file, the *GUI* window will just require to the user to provide the name of the input file and also that of the result files, as shown in Figure 7. For each one of the queries in the list, a result file will be generated with the denomination specified by the user and an index added to the name indicating its position in the list of queries. An example result file is shown in the Figure 8. The content of the result files is another list containing the information about the cases that have been retrieved (as many as demanded by each query).

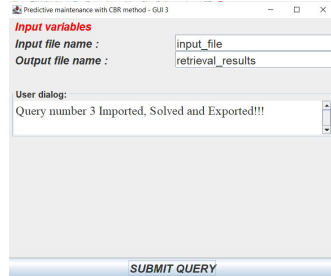


Figure 7: Window of the GUI3

Reference	Sim	Task	Case study type	Case study	Online/Off-line	Input for the	Model Approach	Models	Input type	Number of inq	Performance	Performance	Completeness	Publication identifier
209	0,919	Fault detectio	Rotary machi	Simulated jet-	Online	Time series	Single model	Voting metho	Spinning speed	3	N/A,	N/A,	No info about doi: 10.12700/APH.15.1.2018.2.10	
191	0,901	Fault detectio	Rotary machi	Simulated jet-	Online	Time series	Multi model	Bayes model,	Measurement	2	Robustness ,	I N/A,	10 operations doi: 10.1109/TCST.2013.2177981	
166	0,845	Fault detectio	Rotary machi	Wind turbines	Off-line	Time series	Single model	Gaussian proc	Temperature	22	Accuracy,	<0.945,	one case of s https://doi.org/10.1016/j.renene.2018.10.088	
167	0,845	Fault detectio	Rotary machi	Wind turbines	Online	Time series	Single model	Gaussian proc	Wind power	22	N/A,	N/A,	one case of s https://doi.org/10.1016/j.renene.2018.10.088	
210	0,837	Fault identifi	Rotary machi	Simulated jet-	Online	Time series	Single model	Expert system	Results of a fu	1	N/A,	N/A,	No info about doi: 10.12700/APH.15.1.2018.2.10	
211	0,818	Fault detectio	Rotary machi	Simulated jet-	Off-line	Time series	Multi model	Hybrid Kalma	Compressor T	8	Visual indicat	N/A,	6 operation r doi: 10.1109/ACC.2013.6580667	
192	0,818	Fault identifi	Rotary machi	Simulated jet-	Online	Time series	Multi model	Bayes model,	Measurement	2	Robustness ,	I N/A,	10 operations doi: 10.1109/TCST.2013.2177981	
195	0,786	Fault detectio	Rotary machi	Rotor shafts	Online	Time series	Multi model	Gauss-Markov	Imbalance soi	2	N/A,	N/A,	No info about doi: 10.1117/12.475502	
72	0,772	Fault detectio	Structures	Tank reactor	Online	Time series	Multi model	Updated Rule	Temperature,	3	Probability,	From graphics	2 operational doi: 10.1016/j.ejor.2010.03.032	
98	0,760	Fault detectio	Rotary machi	Steam Turbine	Online	Signals	Single model	Extreme Grad	Condenser va	7	N/A,	N/A,	No info about doi: 10.1016/j.microrel.2015.03.010	
59	0,758	Remaining us	Rotary machi	Aircraft bear	Online	Time series	Multi model	Relevance ve	Health index,	1	Score functio	19,66,	No info about doi: 10.1016/j.ress.2017.12.016	
58	0,758	Remaining us	Rotary machi	Aircraft bear	Online	Time series	Multi model	Ensemble lear	Health index,	1	Score functio	7,8,	No info about doi: 10.1016/j.ress.2017.12.016	
60	0,758	Remaining us	Rotary machi	Aircraft bear	Online	Time series	Single model	Particle filter,	Health index,	1	Score functio	301,8,	No info about doi: 10.1016/j.ress.2017.12.016	
204	0,756	Remaining us	Rotary machi	Wind turbines	Online	Time series	Single model	Geolocation (	Distance, Deg	2	Prognostic ho	(851,00.7)	No info about doi: 10.1016/j.renene.2017.05.020	
2	0,755	Health assess	Rotary machi	Simulated jet-	Off-line	Time series	Single model	Logistic regre	Temperature,	5	Mean absolut	<0.05),	1 operational doi: 10.1016/j.ast.2018.09.044	
1	0,755	Health model	Rotary machi	Simulated jet-	Off-line	Time series	Single model	Logistic regre	Temperature,	5	Mean absolut	<0.05),	1 operational doi: 10.1016/j.ast.2018.09.044	
51	0,754	Remaining us	Rotary machi	Simulated jet-	Off-line	Time series	Multi model	LSTM (Long-S	Temperature,	14	Root mean sq	6,914,7,	6 CMPASS dat doi: 10.1016/j.ast.2019.105423	
53	0,754	Remaining us	Rotary machi	Simulated jet-	Off-line	Time series	Single model	Recurrent Ne	Temperature,	14	Root mean sq	10,245,8,	6 CMPASS dat doi: 10.1016/j.ast.2019.105423	
52	0,754	Remaining us	Rotary machi	Simulated jet-	Off-line	Time series	Single model	LSTM (Long-S	Temperature,	14	Root mean sq	8,245,7,	6 CMPASS dat doi: 10.1016/j.ast.2019.105423	
54	0,754	Remaining us	Rotary machi	Simulated jet-	Off-line	Time series	Single model	Gated recurrence	Temperature,	14	Root mean sq	10,016,0,	6 CMPASS dat doi: 10.1016/j.ast.2019.105423	

Figure 8: Example .csv result file for the GUI3

2.7 The javadoc

A complete *javadoc* has been generated for this project. It is available in the folder *javadoc* of the project. The easiest way to access to the documentation is opening the *index* file in the mentioned folder. It is an *HTML* file in the standard format for *javadoc* resources available in the web, and it may be opened with a web browser. The documentation contains a detailed descriptions of all the classes and methods organized in linked pages, allowing to navigate through the structure of the code. To update the documentation using *Eclipse*, go to *Project* → *Generate Javadoc*. When the window is open, the user must select the folder where the *javadoc* will be saved. It is recommended to use the folder *javadoc*, after having deleted the previous content to avoid problems. Of course, the generation of *javadoc* depends on the *javadoc* format comments that have been added to the code. See [12] for more details on the *javadoc* comments format.

2.8 Management of the *data* folder

The purpose of the *data* folder in the project is to store all the files that are needed for the execution tasks of the application. It is very important to remember that the denomination of the files that are wanted to be used inside this folder must be in coordination with the names introduced in *AppConfiguration*, so as the code is able to find such files. Some of the main types of files and their function are listed:

- Files *.owl* : the ontology files. One important consideration is that these files must be written in the *RDF/XML* syntax (choose that option when using *Save as* in *Protégé*). They can be opened with *Protégé* to visualize and edit the ontology or with a simple plain text editor, which allows to modify manually the statements. Two files may be used, one file should contain a clean ontology (without the declaration of the individuals in the data base) and the other one with all the data base loaded (and a different name, of course). An ontological data base could be directly used for the *myCBB* project build-up by selecting the appropriate option (user text input) when executing the class *myCBBSetting*. In any case, an ontology structure is required, either formed from a *.csv* table or directly provided by the user.
- Files *.csv* : files with a table format. A data base in *.csv* format may be needed to be loaded in the working ontology. The input and result files used by the *GUI3* are also *.csv* format. These files can be opened with *Excel*. Using a plane text editor to open *.csv* files may be recommended to ensure that no weird characters have been introduced by *Excel* at the beginning of the data (an unusual bug in *Excel* that may alter the name of the first column of data).
- Files *.prj* : the project files of *myCBB*, where all the information concerning the query and retrieval must be stored. This type of file can be opened with the *myCBB Workbench* application.
- Files *.ttl* : these files contain the ontology dependencies (BFO, CCO ontologies, etc). They are necessary to read the ontologies when working with local imports, so as the ontology is independent of the online servers. They are available at the GitHub repository [22], and may be updated with newer versions from time to time. For the execution of the code, some mappers are set using *OWL API* to link the *.ttl* files with the *URI* that are used by the ontologies to invoke the corresponding dependencies. For the *OPMAD* ontology, the following files are required: *ArtifactOntology.ttl*, *EventOntology.ttl*, *ExtendedRelationalOntology.ttl*, *GeospatialOntology.ttl*, *InformationEntityOntology.ttl*, *ro-import.ttl*, *TimeOntology.ttl*.
- File *catalog-v001.xml* : this file stores the paths to allow *Protégé* to import the local ontology dependencies in *.ttl* format when opening an ontology file that needs those dependencies.

3 Dependencies

Here are listed the main *APIs* and libraries which are needed for the application, which are included in the *external-lib*s folder of the project:

- *OWL API* : it is an *API* that allows to read, modify, manipulate and create *.owl* ontologies. The version currently used in the application is 5.1.9, but future updates could be possible. The *API* is implemented by including the necessary *.jar* files in the *external-lib*s folder of the application. The last version of *OWL API* is available at the *Maven* repository [2] (artifact owlapi-distribution), but it has been directly obtained as *.jar* files from [1] with all dependencies. Previous versions are also available. A complete documentation can be accessed at [3] and there is an introduction tutorial written by the creators at [6]. The licenses concerning the *API* are the Apache License [4] and the GNU license [5].
- *HermiT* reasoner: it is a reasoner that works in *Protégé*, the most used software for ontology manipulation. There is an implementation for *Java* based on the *OWL API*. The version used is the latest one (1.4.5.519), and it can be found at the *Maven* repository [7] or downloaded from [8] with all dependencies. The reasoner is needed to perform queries on ontologies through *Java*. The documentation for a previous equivalent version (1.3.8.4) is available at [9]. The GNU license is applicable [5].
- *myCBR*: it is an open source tool for case-based reasoning applications. The *Software Development Kit* of *myCBR* project has been implemented in the application with the appropriate *.jar* file. The *myCBR Workbench* application may be very useful for visualizing *.prj* files. All the information concerning *myCBR* project (source code, installation guide, tutorials, javadoc, etc) is available at the website [10].
- *Jena*: it is a tool for ontology manipulation. In particular, it is used in this application for adding the capability to perform *SPARQL* queries on ontologies through *Java* with the *SPARQL* executable class in the project, that uses the *Jena* methods. The latest version is used (4.0.0), and it can be found at the *Maven* repository [13] (artifact jena-arq) or downloaded from [14] with all dependencies. A complete documentation of the *Jena Core* is available at [15]. The Apache license [4] is applicable.
- *SWRL API* with *drools* rule engine: the *SWRL API* may be used to implement *SWRL* rules in an ontology through a *Java* application. Moreover, if the *drools* reasoner is added, the application is able to execute *SQWRL* queries on the working ontology. In this case, the latest version of *SWRL API* (2.0.9) is used, which is available at the *Maven* repository [16] and at [17] for direct *.jar* download with all dependencies. A *javadoc* is available at [18]. Nevertheless, the compatibility of the *SWRL API* with *OWL API* is only guaranteed up to version 4.5.9, even if some later versions could be supported. Moreover, to query an ontology with the *SWRL API* and *SQWRL* language, an additional implementation of a reasoner is necessary: the implementation of *drools* engine is available for this purpose. Once again, it may be

found at the *Maven* repository [19] or for direct *.jar* download with all dependencies at [20].

References

- [1] JAR download (April 2021): <https://jar-download.com/artifacts/net.sourceforge.owlapi/owlapi-distribution>
- [2] MVN repository (April 2021): <https://mvnrepository.com/artifact/net.sourceforge.owlapi/owlapi-distribution/5.1.17>
- [3] OWL API javadoc (April 2021): <https://javadoc.io/doc/net.sourceforge.owlapi/owlapi-distribution/latest/index.html>
- [4] Apache License version 2.0: <https://www.apache.org/licenses/LICENSE-2.0>
- [5] GNU LESSER GENERAL PUBLIC LICENSE: <http://www.gnu.org/licenses/lgpl-3.0.txt>
- [6] MATENTZOGLU Nicolas., PALMISANO Ignazio. *An introduction to OWL API*. University of Manchester, 2016.
<http://syllabus.cs.manchester.ac.uk/pgt/2020/COMP62342/introduction-owl-api-msc.pdf>
- [7] MVN repository (April 2021): <https://mvnrepository.com/artifact/net.sourceforge.owlapi/org.semanticweb.hermit>
- [8] JAR download (April 2021): <https://jar-download.com/?search`box=org.semanticweb.hermit>
- [9] HermiT 1.3.8.4 javadoc (April 2021): <http://javadocx.com/com.hermit-reasoner/org.semanticweb.hermit/1.3.8.4/overview-summary.html>
- [10] myCBR website (April 2021): <http://mycbr-project.org/index>.
- [11] SPARQL syntax reference (April 2021): <https://www.w3.org/TR/sparql11-query/>
- [12] *Javadoc* reference (April 2021): <https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html>
- [13] MVN repository (April 2021): <https://mvnrepository.com/artifact/org.apache.jena/jena-arq/4.0.0>
- [14] JAR download (April 2021): <https://jar-download.com/artifacts/org.apache.jena/jena-arq>
- [15] Jena Core 4.0.0 (April 2021): <https://jena.apache.org/documentation/javadoc/jena/>
- [16] MVN repository (April 2021): <https://mvnrepository.com/artifact/edu.stanford.swrl/swrlapi/2.0.9>
- [17] JAR download (April 2021): <https://jar-download.com/artifacts/edu.stanford.swrl/swrlapi>
- [18] SWRL API Javadoc (April 2021): <http://soft.vub.ac.be/svn-pub/PlatformKit/platformkit-kb-owlapi3-doc/doc/owlapi3/javadoc/overview-summary.html>

- [19] MVN repository (April 2021): <https://mvnrepository.com/artifact/org.apache.jena/jena-arq/4.0.0>
- [20] JAR download (April 2021): <https://jar-download.com/artifacts/org.apache.jena/jena-arq>
- [21] O’CONNOR Martin Joseph, DAS Amar. *SQWRL: A Query Language for OWL*. Stanford Center for Biomedical Informatics Research, 2009.
- [22] GitHub repository (April 2021): <https://github.com/CommonCoreOntology/CommonCoreOntologies>

Summary in French / Résumé en français

Sommaire

D.1	Introduction	202
D.1.1	Maintenance Prédictive	202
D.1.2	Idée conceptuelle et architecture d'un système	202
D.1.3	Questions à la base de ce projet de recherche	203
D.1.4	Organisation du résumé	204
D.2	Vers une approche multimodèle de la maintenance prédictive : une revue littéraire systématique sur le diagnostic et le pronostic	205
D.3	Des besoins et souhaits pour une architecture logique de systèmes de maintenance prédictive	207
D.4	Développement d'une ontologie pour le système de raisonnement par cas	209
D.4.1	Ontologie pour la sélection et l'évaluation des stratégies de maintenance (OMSSA)	209
D.4.2	Ontologie pour la conception de maintenance prédictive (OPMAD)	210
D.5	Développement de systèmes de raisonnement par cas	211
D.5.1	Plateforme MyCBR	212
D.5.2	Représentation des cas	213
D.5.3	Développement du moteur de récupération de composants pour les systèmes de maintenance prédictive	214
D.6	Développer un cadre pour la sélection de modèles de maintenance prédictive	214
D.7	Validation et discussion des résultats	216
D.7.1	Validation croisée	217
D.7.2	Implementation d'un cas d'étude pratique	218
D.8	Conclusion et perspectives de travaux futurs	219
D.8.1	Résumé des contributions	219
D.8.2	Perspectives de travaux futurs	220

D.1 Introduction

D.1.1 Maintenance Prédictive

La maintenance prédictive est une stratégie de maintenance qui vise à déterminer le moment précis pour effectuer des actions de maintenance. Pas trop tôt, car les actions de maintenance peuvent entraîner le changement de pièces qui ont encore une durée de vie restante significative ce qui représente un coût pour les entreprises ; mais pas trop tard non plus, car une panne inattendue peut à son tour donner lieu à une série de conséquences négatives. La maintenance prédictive est une alternative aux autres stratégies de maintenance traditionnelles, telles que la maintenance corrective et la maintenance préventive. Au lieu d'attendre qu'une panne se produise ou d'effectuer une maintenance basée sur des intervalles de fonctionnement fixes, la maintenance prédictive est basée sur le diagnostic de l'état actuel de l'équipement et sur la prédiction du moment où une éventuelle panne pourrait survenir.

La maintenance prédictive est fortement liée à la maintenance basée sur l'état (Condition-Based Monitoring, CBM) et à la gestion de la santé basée sur les prévisions (Prognostics and Health Management, PHM). La maintenance prédictive et ces deux disciplines englobent le diagnostic et le pronostic de maintenance ; parfois ces termes sont même utilisés comme synonymes. Les deux termes maintenance prédictive et CBM sont apparus dans les années 1940 et font référence à la stratégie de maintenance qui vise à anticiper les pannes des machines en fonction de leur état. Cependant, ce n'est qu'au début des années 1990 que cette stratégie prend de l'importance grâce à la mise en place de systèmes de surveillance et d'outils de calcul capables de surveiller les tâches de diagnostic. Quant au pronostic de durée de vie restante, faisant partie de la maintenance prédictive et de la CBM, cela restait une discipline imprécise. Dès le début des années 2000, la discipline PHM a émergé avec l'objectif de couvrir des problèmes de pronostic. Depuis lors, les diagnostics de maintenance et la recherche de pronostics ont suscité beaucoup d'attention de la part des universités et de l'industrie ; grâce à l'augmentation continue de la puissance de calcul et compte tenu des avantages de sa mise en œuvre. Au cours de la dernière décennie, différentes contributions ont été apportées sous des termes différents (maintenance prédictive, CBM et PHM) ; elles se réfèrent au même domaine de recherche.

D.1.2 Idée conceptuelle et architecture d'un système

Selon le manuel INCOSE [INC15], le cycle de vie d'un système peut être divisé en six étapes génériques (voir Figure D.1). Le cycle de vie d'un système commence par l'étape conceptuelle, qui vise à explorer des solutions possibles pour répondre aux besoins et souhaits initiaux des parties prenantes. La phase conceptuelle est suivie de la phase de développement, au cours de laquelle une conception détaillée des composants du système et de leurs interfaces est réalisée. Après la conception détaillée, commence l'étape de production qui est dédiée à la mise en œuvre du système, c'est-à-dire la fabrication, le codage, la création des différents composants et leur intégration finale dans le système. La vérification et la validation du système font partie de la phase de production. Une fois le système validé, il est livré au client pour sa phase d'utilisation. Au stade de l'utilisation, le système remplit la fonction pour laquelle il a été créé. La phase de maintenance se déroule parallèlement à la phase d'utilisation pendant le fonctionnement du système. Cette étape de maintenance est destinée à assurer l'état de fonctionnement optimal du système. En fin de cycle de vie du système, la phase de retrait vise à bien gérer le retrait du système, sa mise hors service, et son démantèlement.

Le processus d'architecture du système se positionne au cœur de la phase conceptuelle. Cette phase commence par rassembler tous les besoins et souhaits des parties prenantes et les formaliser dans les exigences des différents acteurs. Ces exigences sont ensuite utilisées pour créer l'architecture du système

Concept stage	Development stage	Production stage	Utilization stage	Retirement
			Support stage	State

Figure. D.1: Étapes génériques du cycle de vie d'un système (traduit de l'anglais) [INC15]

qui servira de base à la conception détaillée et à la mise en œuvre du système. Le développement de l'architecture peut être divisé en trois niveaux [Roq18]; [INC15] : architecture fonctionnelle, architecture logique et architecture physique. L'architecture fonctionnelle décrit comment les différentes (sous-)fonctions d'un système interagissent les unes avec les autres pour atteindre un objectif spécifique, mais ne fournit aucun détail sur les composants qui remplissent chaque (sous-)fonction. L'architecture logique fournit autant de détails que possible sur les composants de l'architecture et leurs interfaces, mais n'implique pas à ce stade les choix quant aux réalisations et/ou aux technologies spécifiques, ce qui signifie que l'architecture logique montre des composants génériques. L'architecture physique fournit les détails des technologies à affecter à chaque composant logique. L'union de ces trois niveaux d'architecture peut être appelée l'architecture du système et peut alors être définie comme les « concepts ou propriétés fondamentaux d'un système dans son environnement incorporés dans ses éléments, ses relations et dans les principes de sa conception et évolution » [ISO11]. En termes simples, l'architecture traite de la manière dont un ensemble d'éléments, qui peuvent être physiques ou informationnels, sont organisés pour répondre à un objectif spécifique.

D.1.3 Questions à la base de ce projet de recherche

La maintenance prédictive est réalisée à l'aide d'approches spécialisées pour effectuer des tâches de diagnostic et de pronostic, afin de déterminer le bon moment pour déclencher les actions de maintenance. En ce moment, la conception et le développement de ces systèmes spécialisés continuent d'être basés sur des essais et des erreurs. Il existe plusieurs architectures génériques dans les normes et standards, comme [MIM01], mais en réalité la conception d'un nouveau système de maintenance prédictive commence beaucoup plus tôt, au stade conceptuel, moment auquel ces architectures génériques n'aident pas encore l'architecte. En effet, une architecture générique ne fournit pas de liens avec les besoins et souhaits initiaux d'un nouveau système de maintenance prédictive, et ces besoins et souhaits peuvent bien ne pas être couverts par une architecture générique. De plus, une architecture générique ne fournit aucune indication pour sélectionner les technologies appropriées pouvant réaliser les composants génériques [MV19], par exemple pour la détection d'un défaut ou pour l'estimation d'une durée de vie restante à partir d'une série de mesures.

Même si la maintenance prédictive traite en général des tâches de diagnostic et de pronostic, tous les systèmes prédictifs ne couvrent pas exactement les mêmes fonctions. Par exemple, un nouveau système de maintenance prédictive peut être destiné à effectuer des diagnostics sur différents composants d'un système ; plusieurs modules de diagnostic du même type seront nécessaires et les modules de prédiction ne seront pas inclus. Les architectures génériques ne fournissent pas une approche systématique pour les conceptions qui n'ont besoin que d'un sous-ensemble des composants génériques proposés ou lorsque plusieurs composants du même type sont nécessaires.

Il existe un nombre important d'options pour réaliser les fonctions de diagnostic et de pronostic dans un système de maintenance prédictive, et il n'y a pas de directives pour aider l'architecte à sélectionner les modèles, techniques ou algorithmes appropriés qui peuvent exécuter ces fonctions. L'exploration de l'espace de solution pour déterminer les composants appropriés peut alors être complexe et prendre du temps. Cette thèse vise à faciliter le processus architectural des systèmes de maintenance prédictive en permettant une manière plus efficace d'explorer l'espace des solutions et de proposer les composants les plus adaptés à l'architecture du système.

Avant d'aborder la conception de systèmes de maintenance prédictive, il est important de comprendre le domaine de la recherche en maintenance prédictive lui-même et les différentes options disponibles pour effectuer des diagnostics et des pronostics. Les questions de recherche suivantes sont proposées pour guider l'étude de l'état de l'art dans ce domaine :

1. Quelles sont les tendances actuelles du diagnostic et du pronostic en maintenance prédictive ?
2. Quels types de modèles, techniques ou méthodes sont utilisés pour traiter le diagnostic et le pronostic en maintenance prédictive ?
3. Quels sont les principaux défis rencontrés par la maintenance prédictive dans le diagnostic et le pronostic ?

Après l'étude de l'état de l'art, les questions de recherche sont adaptées en fonction des résultats de l'étude et de la motivation initiale de cette recherche liée à la conception de systèmes de maintenance prédictive (voir section D.2).

D.1.4 Organisation du résumé

La section D.2 aborde l'état de l'art de la maintenance prédictive sur la base des questions de recherche initiales introduites ci-dessus. Cette section est liée au chapitre 2 de la thèse, qui est composé d'un article publié qui comprend les tendances actuelles des modèles de diagnostic et de pronostic dans le domaine de la maintenance. La section se termine en affinant les questions de recherche qui ont motivé le reste de la recherche.

La section D.3 propose une approche d'ingénierie des systèmes pour la conception de systèmes de maintenance prédictive, en particulier au stade conceptuel de l'analyse des besoins et souhaits initiaux des parties prenantes jusqu'à la proposition d'une architecture logique. Cette section est liée au chapitre 3 de la thèse qui est composé d'un article publié et présenté lors d'une conférence et se termine en présentant la sélection de la composante maintenance prédictive comme le défi principal qui sera abordé dans les chapitres suivants. Un système d'aide à la décision (Decision Support System, DSS) qui combine des ontologies et un raisonnement basé aux cas est proposé comme une solution possible pour aborder la sélection de composants dans l'approche systématique.

La section D.4 explique l'un des éléments de base du système d'aide à la décision : les ontologies. Des recherches complémentaires ont été spécifiquement menées sur les ontologies qui ont abouti à un article de revue, accepté pour publication au moment de la rédaction de ce manuscrit, et qui présente un modèle ontologique pour la sélection et l'évaluation des stratégies de maintenance (Ontology for Maintenance Strategy Selection and Architecture, OMSSA). Le contexte théorique des ontologies est présenté en soulignant leur importance dans la communauté des chercheurs en raison de leurs capacités à modéliser formellement le vocabulaire d'un domaine spécifique et à le raisonner. La section D.4 explique le développement de l'ontologie qui sera ensuite utilisée dans le DSS pour la sélection des composants.

La section D.5 présente le deuxième pilier du DSS : le raisonnement basé aux cas (Case-Based Reasoning, CBR). Les principes du paradigme de la CBR ont été présentés, y compris les phases de son cycle générique. Dans une première approche pour adresser les questions à la base de cette thèse, le DSS se concentre sur la phase de récupération de la CBR. Une explication de la mise en œuvre du moteur de récupération CBR est fournie à l'aide de code source logicielle.

La section D.6 explique le cadre général d'intégration des composants de base du DSS et comment il s'intègre dans l'approche systématique de la conception des systèmes de maintenance prédictive. Cette

section est liée au chapitre 6 de la thèse qui est composé d'un article de conférence publié et qui est la suite de l'article présenté dans la section D.2. Une validation croisée est effectuée pour démontrer les capacités du DSS.

La section D.7 est destinée à étendre la validation du DSS. Une étude de cas est présentée pour développer l'approche complète proposée dans la recherche actuelle et un exemple de modèle de maintenance prédictive est mis en œuvre pour l'étude de cas sur la base des suggestions du DSS. Les résultats de cette mise en œuvre sont expliqués et analysés.

La section D.8 tire des conclusions de travaux présentés et propose des perspectives pour une suite dans ces activités de recherche.

D.2 Vers une approche multimodèle de la maintenance prédictive : une revue littéraire systématique sur le diagnostic et le pronostic

Une revue systématique de la littérature montre que la maintenance prédictive gagne de plus en plus en importance dans la communauté universitaire, en particulier au cours des 25 dernières années. La Figure D.2 montre le nombre de publications mentionnant les termes « maintenance prédictive », « maintenance conditionnelle » et « pronostic et prise en charge de l'état de la santé » au cours des 25 dernières années dans les sources de recherche consultées.

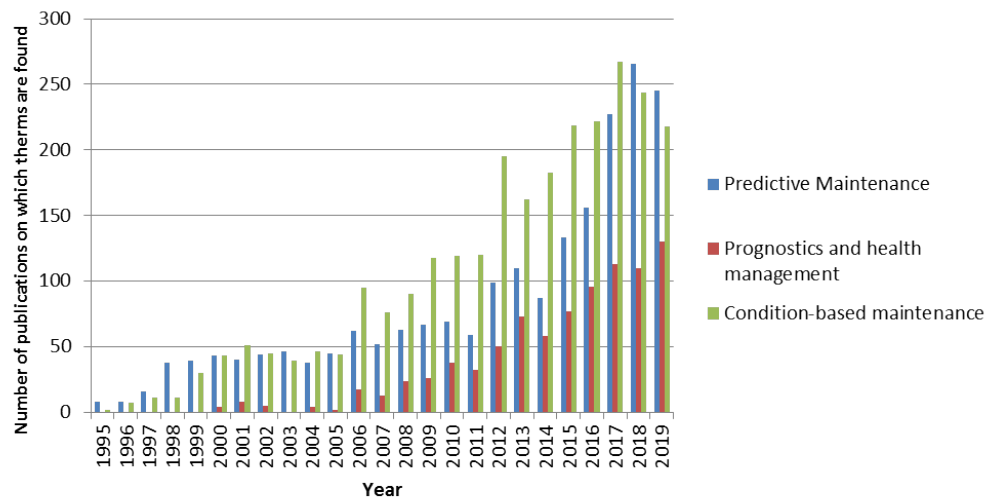
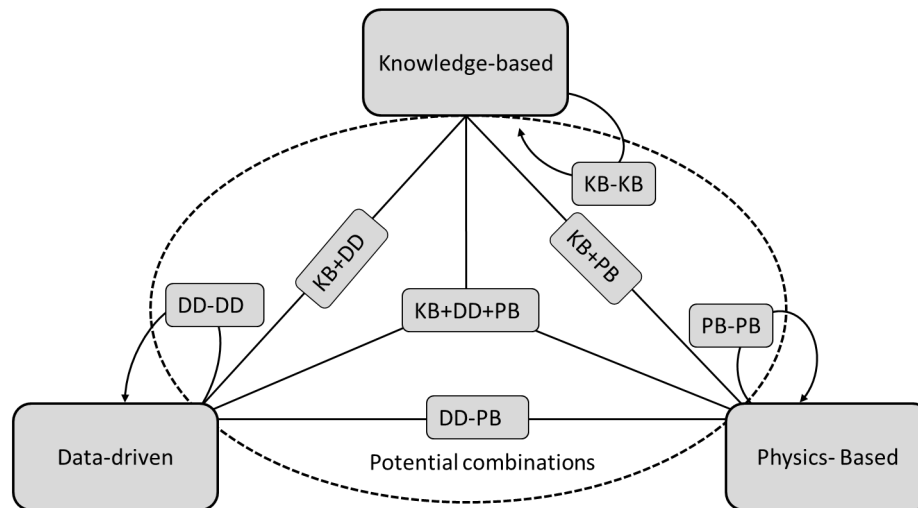


Figure. D.2: Nombre de publications liées à la maintenance prédictive au cours des 25 dernières années

La revue de la littérature se compose de deux parties. La première portait sur les précédentes revues bibliographiques sur la maintenance prédictive, qui ont permis d'identifier les modèles utilisés pour la maintenance prédictive et l'évolution des tendances au cours des années. Les taxonomies utilisées pour classer les différents modèles prédictifs montrent de légères variations terminologiques d'une étude à l'autre dans les études diagnostiques et pronostiques en maintenance. On peut distinguer deux approches principales : les approches mono-modèles et les approches multi-modèles. Pour les approches mono-modèles, on peut identifier trois familles de modèles : modèles basés sur la connaissance, modèles basés sur des données et modèles basés sur la physique. Les approches multi-modèles combinent au moins deux modèles des trois familles de modèles mentionnées. Les approches multi-modèles peuvent avoir différentes configurations et sont parfois appelées modèles hybrides.

La seconde partie de la revue de la littérature a été consacrée à l'étude des tendances actuelles des différentes approches des modèles. La revue a démontré l'existence d'une tendance à la mise en œuvre d'approches multi-modèles puisqu'un seul modèle souvent ne satisfait pas à toutes les fonctions d'un système de maintenance prédictive. La figure D.3 montre toutes les possibilités de combinaison de modèles dans des approches multi-modèles.



KB: Knowledge-based model. DD: Data-driven model. PB: Physics-based model.

Figure. D.3: Combinaisons possibles de modèles de maintenance prédictive

La revue bibliographique a permis d'identifier les enjeux actuels de la maintenance prédictive :

- Extrapolation des solutions de maintenance prédictive existantes dans des applications complexes, incluant plusieurs composants et leurs défaillances associées. La plupart des applications identifiées se concentraient sur un seul composant avec un nombre limité de défauts possibles. Cependant, les applications réelles sont souvent des systèmes complexes constitués de nombreux composants et de nombreux défauts associés à chaque composant et au système lui-même. Les approches multi-modèles offrent une solution potentielle pour surmonter la complexité des systèmes de maintenance prédictive.
- L'absence d'une approche systématique pour concevoir et développer des systèmes de maintenance prédictive. Il existe des standards, des normes et des architectures génériques pour développer de nouveaux systèmes de maintenance prédictive, tels que OSA-CBM. Cependant, ils se concentrent uniquement sur les composants fonctionnels de base du système et ne couvrent pas les aspects importants des indicateurs de performance ou des contraintes du contexte du système. De plus, ils n'offrent pas d'explication cohérente sur les modèles à utiliser en fonction des besoins initiaux du système de maintenance prédictive. L'absence d'approche systématique limite la mise en œuvre de systèmes de maintenance prédictive dans les applications industrielles à grande échelle.
- La fusion de diverses sources de données de contrôle de l'état des systèmes. Ce défi est lié à l'extrapolation des modèles actuels de maintenance prédictive aux systèmes complexes. Les systèmes techniques peuvent avoir différents types de sources de données, par exemple des mesures de capteurs, des enregistrements de maintenance, des enregistrements opérationnels, des documents de conception, etc. Des informations importantes pourraient être recueillies auprès de toutes ces sources pour mettre en œuvre de nouveaux systèmes de maintenance prédictive.

- Intégration des données d'influence externe à l'exploitation des systèmes. Le fonctionnement des systèmes peut varier en fonction du contexte d'exploitation. Des changements dans ce contexte opérationnel peuvent affecter directement les performances du système et, par conséquent, les analyses de surveillance de l'état de santé du système. Il peut ainsi déclencher de fausses alarmes suggérant l'existence de défauts, ou il peut à l'inverse empêcher l'identification de défauts existants. Des modèles complémentaires capables d'intégrer une influence externe à des fins de maintenance prédictive pourraient contribuer à une solution.
- Gestion de l'incertitude numérique des mesures. L'incertitude de mesures et de calculs affecte directement l'exactitude du diagnostic et du pronostic. Cela peut être dû aux données collectées ou aux imperfections du modèle utilisé pour l'analyse. Cela peut affecter la fiabilité des résultats. La gestion de l'incertitude est vitale pour les systèmes critiques soumis à la réglementation des autorités. C'est le cas des systèmes critiques tels que les centrales nucléaires et les avions où les réglementations sont restrictives pour maintenir les normes de sécurité et éviter les événements catastrophiques.

L'étude de l'état de l'art a motivé l'amélioration des questions de recherche. Le reste de la recherche a été motivé par les questions suivantes :

1. Comment aborder l'architecture et la conception des systèmes de maintenance prédictive?
2. Comment sélectionner un modèle ou une combinaison de modèles appropriés compte tenu d'un nouveau problème de maintenance prédictive à résoudre?
3. Comment proposer une approche adaptée pour une solution de maintenance prédictive?
4. Comment un concepteur peut-il bénéficier de l'expérience des systèmes existants pour développer de nouvelles solutions de maintenance prédictive?

D.3 Des besoins et souhaits pour une architecture logique de systèmes de maintenance prédictive

La question de recherche No. 1 obtenue à l'issue de l'étude de l'état de l'art porte sur la conception de nouveaux systèmes de maintenance prédictive. Dans l'énoncé de recherche présenté dans la section D.1, il a été expliqué que la conception et le développement de systèmes de maintenance prédictive sont toujours basés sur des essais et des erreurs. Malgré l'existence de plusieurs architectures génériques dans les normes et standards, telles que [MIM01], l'étape conceptuelle de tels systèmes n'est pas entièrement couverte. Il existe un écart dans le développement de tels systèmes, depuis la collecte des besoins et des souhaits pour le nouveau système jusqu'à la création de l'architecture qui répond aux besoins et souhaits initiaux.

La création d'un nouveau système doit toujours commencer par une analyse des besoins et souhaits exprimés (ou pas) par les parties prenantes. Ces besoins et souhaits permettent d'établir une liste des exigences des parties prenantes pour le nouveau système. Les recherches actuelles proposent une approche d'ingénierie des systèmes pour couvrir l'étape conceptuelle des systèmes de maintenance prédictive.

La Figure D.4 montre les différentes étapes abordées dans l'approche d'ingénierie des systèmes proposée dans cette thèse. Il commence par recueillir les besoins et les souhaits initiaux des parties prenantes, qui sont par la suite traduits dans une liste formelle des exigences des parties prenantes. Les exigences sont hiérarchisées et classées en exigences fonctionnelles, de performance, structurelles et expérientielles. Cette classification aidera à la création de l'architecture du système. Les exigences fonctionnelles sont utilisées

pour démarrer le processus d'architecture, en particulier pour effectuer l'analyse fonctionnelle et créer le fonctionnel. L'architecture fonctionnelle est ensuite utilisée pour développer l'architecture logique qui reste générique. Les exigences de performance, structurelles et expérientielles sont utilisées pour faire la sélection des composants pour répondre à l'architecture logique et créer l'architecture physique.

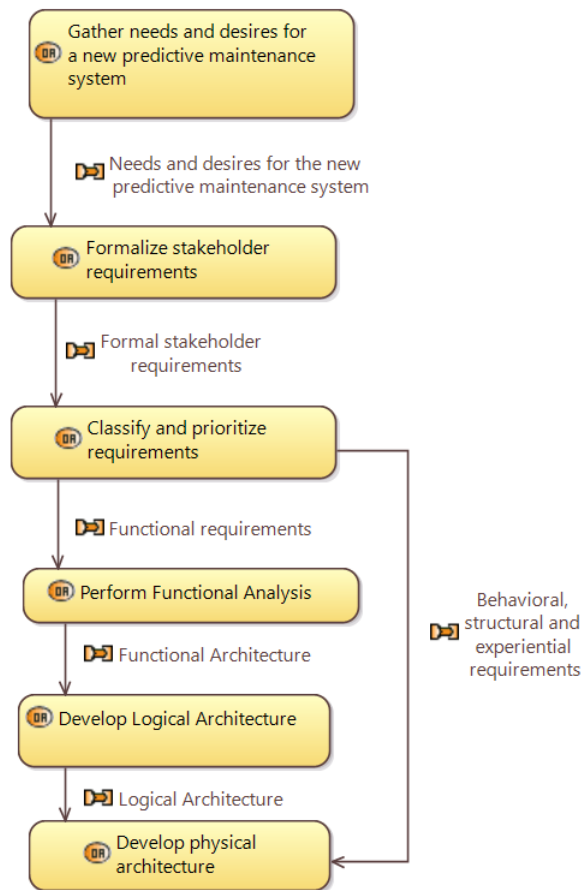


Figure. D.4: Étapes abordées dans l'approche d'ingénierie des systèmes pour la conception de systèmes de maintenance prédictive

Cette section est liée au chapitre 3 de la thèse dans lequel une approche systématique a été proposée pour aborder l'étape conceptuelle des systèmes de maintenance prédictive afin de répondre à la première question de recherche affinée. Les différentes étapes depuis la satisfaction des besoins et souhaits initiaux des parties prenantes jusqu'à la définition de l'architecture logique ont été couvertes et différentes méthodes ont été proposées pour y répondre. Cependant, le reste des questions de recherche affinées n'a pas encore trouvé de réponse. En maintenance prédictive, l'espace des solutions est vaste et complexe comme le montre le chapitre 2 de la thèse. Il n'y a pas de règles spécifiques qu'un architecte peut suivre pour sélectionner les bons modèles et approches pour résoudre de nouveaux problèmes de maintenance prédictive. Dans cette recherche, une hypothèse est proposée pour surmonter ce problème : la mise en œuvre d'un système d'aide à la décision (DSS) basé sur le raisonnement basé au cas (CBR) et soutenu par des ontologies pourrait aider l'architecte à sélectionner des composants appropriés sur la base d'expériences passées à partir d'expériences réussies.

D.4 Développement d'une ontologie pour le système de raisonnement par cas

Le DSS est composé de deux blocs principaux : le raisonnement basé aux cas (CBR) et les ontologies. La CBR est un paradigme de raisonnement qui cherche à résoudre de nouveaux problèmes sur la base des expériences de problèmes similaires résolus dans le passé. La CBR est abordée dans le chapitre 5 de la thèse et dans la section D.5 de ce résumé, mais une brève introduction est nécessaire pour comprendre le rôle que jouent les ontologies dans le DSS développé dans les recherches actuelles. Les systèmes CBR sont développés sur un vocabulaire de base [Alt+12]; [Sán+12]. Ce vocabulaire est nécessaire pour structurer les cas de problèmes résolus stockés dans un « case base », pour les mesures de similarité qui comparent le nouveau problème avec ceux du « case base », et pour les connaissances nécessaires pour adapter la solution récupérée au nouveau problème (voir Figure D.5). Pour les besoins de cette thèse, une ontologie est choisie pour servir de cadre terminologique (vocabulaire de base). Un modèle d'ontologie fournit les termes, les définitions et les relations entre les termes qui sont utilisés pour construire la structure de cas, les similitudes et les connaissances adaptatives dans le DSS. Ce chapitre est consacré au contexte théorique et au développement du modèle d'ontologie pour le DSS proposé.

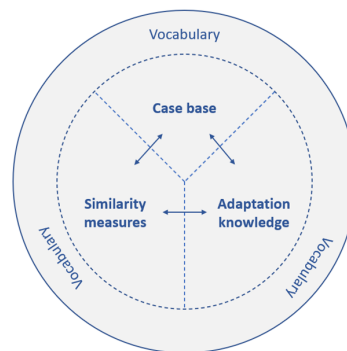


Figure. D.5: Raisonnement basé aux cas, développé sur un cadre de vocabulaire [Alt+12]

En sciences de l'information, une ontologie est une description explicite formelle de concepts dans un domaine de discours, propriétés de chaque concept qui décrivent ses caractéristiques, attributs et restrictions [NM01]. L'un des objectifs les plus courants dans le développement d'ontologies est de « partager une compréhension commune de la structure des informations entre les personnes et les agents logiciels » [Gru93]; [Mus92]. Cela signifie que le vocabulaire utilisé par les personnes dans un domaine de connaissance spécifique peut être « lisible par machine ». Tous les concepts d'une ontologie sont représentés par des classes liées par des propriétés (également appelées relations). Les ontologies sont développées avec des langages formels. L'un des plus reconnus est le langage d'ontologie Web dans sa deuxième version (OWL2) qui est pris en charge par le World Wide Web Consortium (W3C) [Wor12].

D.4.1 Ontologie pour la sélection et l'évaluation des stratégies de maintenance (OMSSA)

Une enquête complémentaire au fil conducteur de cette thèse a permis la création d'un modèle ontologique pour la sélection et l'évaluation des stratégies de maintenance (OMSSA). La création d'OMSSA suit les normes les plus élevées et les dernières tendances en matière de création d'ontologies à l'aide d'ontologies de référence de haut et de moyen niveau. Cela facilitera la réutilisation et l'intégration d'OMSSA avec d'autres ontologies connexes. La contribution de l'OMSSA a été consolidée dans un article scientifique qui

a été accepté pour publication au moment où cette thèse a été écrite. OMSSA sert de base à la création du modèle ontologique utilisé pour le système d'aide à la décision (DSS) proposé dans cette thèse.

D.4.2 Ontologie pour la conception de maintenance prédictive (OPMAD)

OPMAD est une extension de l'OMSSA et les mêmes normes et méthodologies ont été suivies pour son développement. Dans ce qui suit dans ce résumé, les noms de classe d'OPMAD seront conversés en anglais afin de maintenir la cohérence avec le modèle et les graphiques. La Figure D.6 présente les classes et relations les plus importantes dans OPMAD. En raison des limitations d'espace, les sous-classes ne sont pas représentées sur la figure. Comme l'objectif d'OPMAD est d'aider le DSS à identifier des modèles appropriés pour les systèmes de maintenance prédictive, l'explication des classes d'ontologies et de leurs relations commence à partir de la classe Modèle de maintenance prédictive (*Predictive Maintenance Model*) et est basée sur les termes présentés dans la Figure D.6. Un modèle de maintenance prédictive est transporté dans un module de maintenance prédictive (*Predictive Maintenance Module*) qui est un composant d'un système de maintenance prédictive (*Predictive Maintenance System*). Chaque module de maintenance prédictive a une fonction de maintenance prédictive (*Predictive Maintenance Function*), cette recherche se concentre sur les fonctions de diagnostic et de pronostic.

Le modèle de maintenance prédictive intégré dans le module est directement lié à un élément maintenable (*Maintainable item*), qui est la classe qui décrit le système maintenable pour lequel le système de maintenance prédictive est développé. L'élément maintenable est classé par la classe de type d'élément maintenable (*Maintainable item type*) qui a été ajoutée à des fins de calcul de similarité dans le DSS ; ces types de classes aident à comparer le nouveau problème à traiter avec le problème résolu dans un « case base » d'un système CBR. Les éléments maintenables appartenant au même type partagent des caractéristiques de dégradation importantes et donc les mêmes modèles de maintenance prédictive peuvent être proposés pour résoudre les mêmes fonctions de maintenance prédictive. L'élément maintenable a sa propre fonction (*Function*) qui est affectée par une défaillance (*Failure*) se manifestant par un mode de défaillance (*Failure Mode*). Un élément maintenable a un ou plusieurs modes de défaillance qui font également l'objet du modèle de maintenance prédictive. Dans le modèle de maintenance prédictive sont entrées les données d'état (*Condition*) qui sont utilisées pour effectuer des diagnostics et des pronostics. Les deux qualités importantes pour la mise en œuvre d'un modèle de maintenance prédictive sont son type et sa configuration. Le type de modèle de maintenance prédictive (*Predictive Maintenance Model Type*) classe le modèle en familles de modèles basés sur les connaissances, sur les données et sur la physique. Ce type de classification est utile aux fins de la similitude et des préférences dans le DSS.

La configuration du modèle de maintenance prédictive (*Predictive Maintenance Model Configuration*) indique si un modèle doit être complété par d'autres modèles pour remplir sa fonction ; il peut améliorer les performances, mais augmente la complexité du développement. Le module de maintenance prédictive est noté par indicateur de synchronisation (*Module Synchronization*) et de performance (*Performance indicator*). Ces deux qualités fournissent des informations utiles sur la façon dont les modèles de maintenance prédictive intégrés dans les modules doivent être testés et synchronisés avec l'élément maintenable. Toutes ces informations sur les modèles de maintenance prédictive, les éléments maintenables, leurs modèles de défaillance, entre autres, sont collectées dans le cas de la maintenance prédictive (*Predictive Maintenance Case*) qui est documentée et publiée via un article de maintenance prédictive (*Predictive Maintenance Article*), un type spécial de l'article qui se concentre sur la maintenance prédictive. A partir de ces articles de maintenance prédictive, certains indicateurs bibliométriques sont également inclus dans l'ontologie qui sera utilisée dans le système CBR. Ces indicateurs bibliométriques sont fournis en tant qu'attributs de solution afin que l'ingénieur puisse facilement trouver la source d'information pour un cas spécifique. Le titre, l'identifiant et l'année de publication de l'article peuvent être utilisés à des fins de similarité et/ou comme source d'information pour plus de détails sur les modèles de maintenance

```

graph TD
    PdM_System[PdM System] -- hasPart --> PdM_Module[PdM Module]
    PdM_System -- hasQuality --> Synchronization1[Synchronization]
    PdM_Module -- hasFunction --> PdM_Module_function[PdM Module function]
    PdM_Module -- isCarrierOf --> PdM_Model[PdM Model]
    PdM_Module -- hasConfiguration --> PdM_Model_configuration[PdM Model configuration]
    PdM_Model -- hasType --> PdM_Model_type[PdM Model type]
    PdM_Model -- hasInput --> Condition_Data[Condition Data]
    PdM_Model -- isAbout --> Maintainable_item[Maintainable item]
    Condition_Data -- hasType --> Condition_Data_Type[Condition Data Type]
    Maintainable_item -- hasType --> Maintainable_item_type[Maintainable item type]
    Maintainable_item -- hasCaseStudy --> PdM_Case[PdM Case]
    Maintainable_item -- hasFailureMode --> Failure_Mode[Failure Mode]
    PdM_Case -- isCarrierOf --> PdM_Article[PdM Article]
    Failure_Mode -- describes --> Failure[Failure]
    PdM_Article -- isAbout --> PdM_Article_title[PdM Article title]
    PdM_Article -- isAbout --> PdM_Article_publication[PdM Article Publication year]
    PdM_Article_title -- isAbout --> PdM_Article_publication
    PdM_Module_function -- is a --> Function[Function]
    Failure -- isAffectedBy --> Function
    Function -- hasFunction --> Maintainable_item
  
```

D.5 Développement de systèmes de raisonnement par cas

Le cycle CBR démarre lorsqu'un nouveau problème est présenté. La première phase vise à récupérer les cas les plus similaires à partir d'une base de connaissances qui stocke tous les cas précédents. Le cas cible (nouveau) est comparé aux cas existants dans la base de connaissances à l'aide de différentes mesures de similarité. Le cas récupéré le plus proche est proposé comme solution possible en phase de réutilisation. Une certaine adaptation peut être nécessaire pour appliquer la solution dans le cas prévu. Une fois la solution suggérée et mise en œuvre, la phase d'examen est effectuée. Si la solution suggérée parvient à résoudre le problème, elle est confirmée et dans la dernière phase, elle est conservée dans la base de connaissances afin qu'elle puisse être réutilisée dans de futurs problèmes similaires. Plus de détails sur chaque phase de la

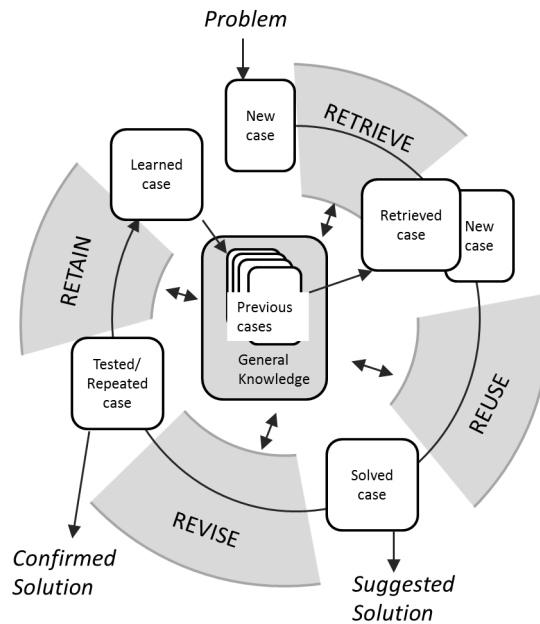


Figure. D.7: Cycle de raisonnement basé aux cas, image inspirée par [AP94]

CBR sont fournis dans les sous-sections suivantes.

D.5.1 Plateforme MyCBR

Le module CBR du DSS dans ce projet de recherche a été créé à l'aide de myCBR, un outil de recherche pour le raisonnement basé aux cas « open source » [Alt+12]. Lors du développement d'un système de récupération CBR à l'aide de myCBR, la première étape consiste à déterminer les attributs des cas. Les cas sont représentés par un vecteur d'attributs caractérisant les problèmes et les solutions :

$$\text{Case} = [\text{Attributs de problème}, \text{Attributs de solution}]$$

Les attributs de problème sont utilisés pour mesurer la similarité d'un cas cible et des cas d'un « case base ». Les attributs de la solution, comme son nom l'indique, fournissent des informations pertinentes relatives à la solution du problème dans chaque cas stocké dans la base de cas.

La deuxième étape consiste à attribuer une similarité locale à chaque attribut du problème. MyCBR fournit plusieurs options pour calculer la similarité pour chaque attribut du problème. Parmi les options disponibles, trois fonctions de similarité ont été utilisées pour les recherches actuelles :

1. Similarité nombre entier/flottant : cette fonction de similarité est utilisée pour les attributs numériques. La similarité est obtenue par une différence entre une valeur de référence et les valeurs d'entrée de la fonction. Une fonction mathématique est nécessaire pour définir comment la similarité diminue à mesure que les valeurs d'entrée s'éloignent de la valeur de référence. Cette fonction mathématique peut être linéaire, exponentielle ou déterminée par des points discrets sur le plan cartésien.

2. Similitude de symbole : Cette mesure de similarité est recommandée pour les variables avec un ensemble fixe d'options. Ces options sont organisées dans une matrice de similarité et certaines valeurs numériques sont données pour établir la similarité entre les options. Cette fonction de similarité a été modifiée afin que certains attributs de la recherche actuelle interagissent avec le modèle ontologique. Les valeurs de la matrice de similarité sont obtenues automatiquement à partir des classes et relations d'une ontologie en utilisant l'approche de similarité basée sur les caractéristiques proposées par [Sán+12].
3. Similitude de chaîne de caractères : la similarité d'attribut est obtenue sur la base de chaînes de texte ouvertes. Contrairement à la similarité de symboles qui a un nombre limité d'options pour définir la similarité, la similarité de chaîne n'a que la restriction d'avoir des phrases ou des mots en entrée. MyCBR propose trois options pour calculer la similarité basée sur les chaînes : Equality, Ngram et Levenshtein. Pour l'étude de cas actuelle, la fonction Levenshtein a été sélectionnée pour calculer les attributs en fonction des chaînes de similarité. La fonction Levenshtein offre un moyen flexible de calculer la similarité en fonction de chaque caractère de la chaîne [Lev66]. Ceci est particulièrement utile lorsqu'il existe un grand nombre d'options inconnues lors de la création de similitudes, et il tolère également les fautes d'orthographe mineures.

Après le calcul de similitude pour chacun des attributs du problème, ces similitudes individuelles sont combinées pour en dériver une similitude globale. Chaque attribut a un poids et à travers une fonction d'agrégation la similarité globale est calculée. L'objectif des poids est de donner une importance particulière aux attributs du problème. Aux fins de cette recherche, tous les attributs reçoivent un poids de 1, ce qui signifie qu'ils sont d'égale importance. MyCBR propose deux options différentes pour calculer la similarité globale : somme pondérée et distance euclidienne.

- Somme pondérée : comme son nom l'indique, c'est la somme des similitudes compte tenu du poids de chacun.
- Distance euclidienne : la distance euclidienne entre deux points dans l'espace euclidien est un nombre, la longueur d'un segment de droite entre les deux points.

Dans cette thèse, les deux fonctions de fusion peuvent être choisies par l'utilisateur. Il est important de noter que seuls les attributs retenus dans la description du problème sont pris en compte pour calculer la similarité globale. Cela signifie que si l'architecte n'a que deux ou trois attributs sur les sept qui décrivent le problème, la similarité globale sera calculée en fonction de ces deux ou trois attributs.

D.5.2 Représentation des cas

« Un cas est une connaissance dans un contexte particulier qui représente une expérience qui enseigne une leçon essentielle pour atteindre le but du raisonneur » [Kol93]. Les cas sont souvent représentés sous une forme couplée [problème, solution], dans laquelle les similitudes sont appliquées à la partie "problème" afin que la partie "solution" soit récupérée. La représentation du cas est composée de trois parties principales [BKP05] :

1. Définissez les attributs du cas.
2. Définissez la structure du contenu du cas.
3. Organisez le « case base ».

La représentation et la structure du cas se font à l'aide d'OPMAD. La Figure D.6 montre les différents attributs (classes en ontologie) d'un cas représenté dans OPMAD. Ces classes d'ontologies peuvent être divisées en deux groupes différents : les attributs de problème et les attributs de solution :

- **Attributs du problème**=[PdM Function,Maintainable Item,Maintainable Item Type, Condition Data Type,Module synchronization,PdM Article Publication year]
- **Attributs de la solution**=[PdM Model,PdM Model Configuration,PdM Model Type,Module Performance Indicator,PdM Article Identifier,PdM Article Title]

D.5.3 Développement du moteur de récupération de composants pour les systèmes de maintenance prédictive

Le « case base » dans le DSS est une version instanciée de l'ontologie OPMAD qui a été renseignée de cas réussis d'implémentations de maintenance prédictive à partir d'une revue bibliographique approfondie. Le processus d'instanciation d'OPMAD comprend la définition des différentes variables de recherche et les options possibles pour chaque variable ; cela permet une meilleure structure du « case base » et facilite la récupération des cas pendant le raisonnement. Pour chaque attribut de problème, une similarité locale est sélectionnée parmi les options possibles : entier, symbole, basé sur une ontologie, texte ouvert. Le Tableau D.1 montre les fonctions de similarité attribuées à chacun des attributs du problème. Deux fonctions d'agrégation différentes ont été ajoutées pour calculer la similarité globale entre un cas cible et les cas stockés dans le « case base ». Une interface graphique a été développée pour faciliter la vérification et la validation du moteur de récupération. Le moteur de récupération développé est le cœur du DSS pour la sélection des composants de maintenance prédictive. Le chapitre suivant est orienté pour montrer le cadre complet du système CBR avec une ontologie pour la sélection des composants de maintenance prédictive. Les détails du développement du moteur de récupération sont fournis dans le guide de code à l'annexe C.

Tableau D.1: Affectation des fonctions de similarité

Attribute	Similarity function
PdM Function	Symbole (Ontologie)
Maintainable item	Chaîne (Levenshtein)
Maintainable item type	Symbole (égalité)
Condition Data	Symbole (Ontologie)
Condition Data Type	Symbole (égalité)
Module synchronization	Symbole (égalité)
PdM Article Publication Year	Entier (fonction définie par des points)

D.6 Développer un cadre pour la sélection de modèles de maintenance prédictive

Une étape commune dans le développement de l'architecture des systèmes concerne l'architecture logique. L'architecture logique fournit autant de détails que possible tout en conservant les composants génériques. Ces composants génériques doivent être remplacés par des composants spécifiques créant ainsi l'architecture physique du système. Pour la sélection des composants, il est possible d'appliquer la créativité structurée. La créativité structurée dans l'architecture logique est basée sur l'analyse des combinaisons possibles des différents composants physiques/informatifs qui peuvent être sélectionnés pour répondre à chaque

composante logique. Cela permet d'explorer l'espace des solutions et peut aider l'architecte à identifier des solutions innovantes pour un nouveau système. Si plusieurs solutions possibles sont identifiées, une analyse de compromis peut être nécessaire pour sélectionner la plus appropriée. L'architecture choisie sert de base à la conception détaillée des systèmes. Les connaissances acquises grâce au nouveau système mis en œuvre peuvent être utilisées par l'architecte pour développer de futurs systèmes. Il existe une analogie entre le travail de créativité structuré effectué par l'architecte pour sélectionner les composants appropriés pour répondre à l'architecture logique et les quatre phases du raisonnement basé aux cas : récupérer, réutiliser, réviser et conserver. Une représentation graphique de cette analogie est présentée à la Figure D.8 (en notation Capella [Roq18]). L'analogie proposée relie la phase de récupération CBR à la recherche de l'architecte dans les systèmes précédents connexes qui peuvent servir d'inspiration pour le développement du nouveau système.

- La phase de réutilisation du CBR serait l'affectation des composants identifiés pour se conformer à la nouvelle architecture du système.
- La phase d'examen CBR commencerait par l'analyse de compensation effectuée par l'architecte pour identifier les composants les plus appropriés. Cette phase se poursuit jusqu'à la vérification et la validation du système mis en œuvre, qui sont généralement réalisées par des acteurs autres que l'architecte.
- Après la validation du nouveau système, l'architecte peut conserver les enregistrements qui seront utilisés à l'avenir ; ce serait la phase de rétention CBR.

Cette analogie est générique en termes du système que l'architecte va développer. Toutes les activités sont effectuées « manuellement » par l'architecte et les enregistrements des expériences précédentes peuvent ne pas être structurés pour faciliter leur réutilisation. Cette approche manuelle peut fonctionner lorsque le nombre d'expériences précédentes est limité afin que l'espace de solution à explorer par l'architecte reste gérable. Lorsque le nombre d'expériences précédentes est élevé ou que les exigences sont complexes, la sélection des composants les plus adaptés peut ne pas être facile. Reprendre connaissance d'un nombre important d'architectures plus anciennes peut prendre du temps et des options importantes peuvent être perdues.

Compte tenu de cette analogie entre la CBR et le travail de créativité structuré, différents algorithmes peuvent être proposés pour accompagner l'architecte dans les différentes phases du travail architectural. Dans une première étape, la recherche actuelle s'est concentrée sur le développement d'un système d'aide à la décision (DSS) capable de rechercher et de recommander des composants physiques / informationnels appropriés. Cela peut aider l'architecte à gagner du temps dans la phase de conception, et permet une analyse plus large effectuée par des machines, analyses qui peuvent difficilement être approchées par l'homme.

La Figure D.9 présente un concept de DSS et comment il s'intègre dans l'analogie présentée à la Figure D.8. Ce concept est composé de trois parties principales : la base de données «le case base », le moteur de recherche et l'ontologie spécifique au domaine. La base de données de cas stocke les cas passés dans un format structuré pour une réutilisation facile. L'architecte présentera les informations du nouveau système en cours de développement au moteur de récupération et récupérera, dans le case base, un ensemble de cas les plus similaires qui servira d'inspiration lors de la sélection des composants appropriés pour l'architecture logique.

Au sein de cette structure, l'ontologie joue un rôle essentiel. Les cas, les attributs de ces cas et les similitudes entre les différentes variables textuelles sont souvent décrits en langage naturel. Modéliser ce langage naturel et le rendre lisible par machine est nécessaire pour automatiser la recherche de cas. L'intégration de l'ontologie proposée et du module de récupération CBR est expliquée plus en détail dans

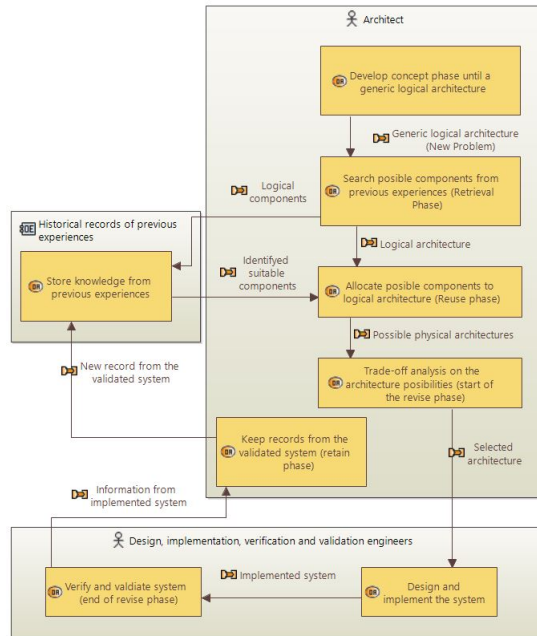


Figure. D.8: Analogie entre le raisonnement basé au cas et les tâches effectuées par un concepteur de systèmes, en notation Capella [Roq18]

une enquête complémentaire présentée à l'annexe B. La section suivante traite de la validation du DSS proposé et de la discussion des résultats obtenus.

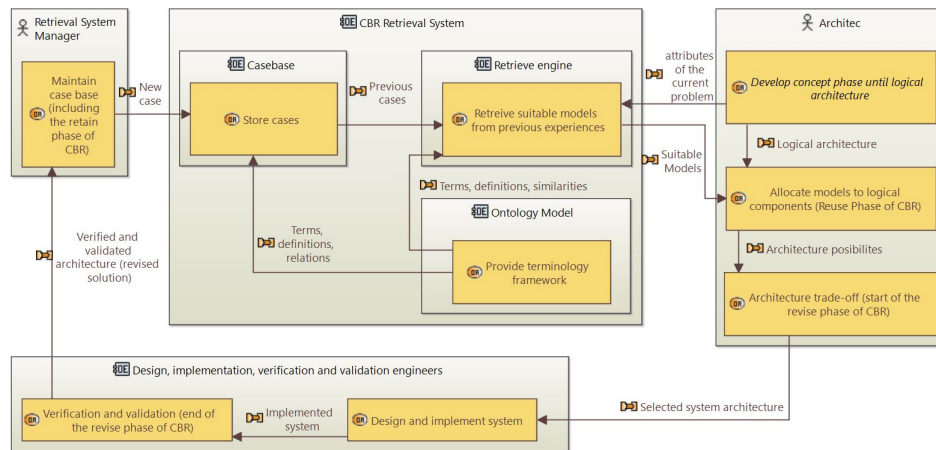


Figure. D.9: Idée conceptuelle d'incorporation de DSS pour la sélection des composants du système

D.7 Validation et discussion des résultats

La validation du DSS est divisée en deux parties : une validation croisée pour confirmer la cohérence du DSS et une mise en œuvre d'un modèle suggéré dans une étude de cas pour tester les recommandations du DSS dans un exemple pratique.

D.7.1 Validation croisée

La validation croisée est une technique qui peut être utilisée pour tester l'efficacité de modèles d'intelligence artificielle entraînés. Pour l'approche actuelle, une approche fractionnée de données de test et d'apprentissage est retenue pour effectuer une validation croisée. Pour l'ensemble de données, 63 des 263 cas dans le « case base » ont été tirés au sort pour les tests, laissant le reste comme ensemble d'apprentissage. Pour chacun des 63 cas tiré au sort, les attributs du problème ont été présentés au DSS à l'aide de l'interface graphique. Puis, l'attention s'est portée sur les 10 cas les plus similaires lors de la récupération. La Figure D.10 montre l'exemple d'un test de récupération dans lequel tous les attributs de problème dans le cas cible correspondent aux mêmes attributs dans le cas 35. Cette correspondance complète entre les attributs entraîne une similarité globale de 1.

Predictive maintenance with CBR method - GUI 2

Input variables

PdM function : Remaining useful life estimation

Maintainable item type : Rotary machines

Maintainable item : Rolling bearings

Condition data type : Vibrations

Module synchronization : Off-line

Condition data : Time series

Variable weights

Remaining useful life estimation	1.0
Rotary machines	1.0
Rolling bearings	1.0
Vibrations	1.0
Off-line	1.0
Time series	1.0

Additional inputs

Number of cases to retrieve: 10

Aggregation function to use: euclidean

User dialog:

I found Case35 with a similarity of 1.000 as the best match. The 10 best cases shown in a table:

Case	Description
Case35 Sim = 1.000	Reference, Similarity and Input variables Reference: 35 Task: Remaining useful life estimation Case study type: Rotary machines Case study: Rolling bearings Online/Off-line: Off-line Input for the model: Time series Models: Convolutional Neural Network Input type: Vibrations Publication Year: 2018 Publication identifier: DOI: 10.1109/ACCESS.2018.2804930

SUBMIT QUERY

Figure. D.10: Exemple de reprise de dossier à l'aide de l'interface DSS

L'espace de solution des modèles de maintenance prédictive peut être divisé en deux groupes comme expliqué dans [Mon+20]. Le premier groupe est composé d'approches à modèle unique, divisées en trois catégories principales : les modèles basés sur la connaissance, les modèles basés sur les données et les modèles basés sur la physique. Le deuxième groupe est constitué d'approches multi-modèles, dans lesquelles au moins deux modèles de l'une des trois catégories mentionnées sont combinés pour réaliser une fonction spécifique du système de maintenance prédictive. La maintenance prédictive est composée de tâches de diagnostic et de pronostic. Selon la revue de la littérature [Mon+20], les tâches de diagnostic telles que la détection des défauts, l'identification des défauts et la modélisation de l'état de santé peuvent être abordées par des modèles des trois catégories d'approches à modèle unique ou d'approches multi-modèles. En revanche, pour les tâches de prédiction, il est difficile de trouver des modèles basés sur les connaissances. Par exemple, les fonctions de prédiction, telles que l'estimation de la durée de vie utile restante et les fonctions de prévision de l'état suivant, sont généralement réalisées par des modèles basés sur la physique, des modèles basés sur les données et des approches multi-modèles qui combinent des modèles basés sur la physique et des données. La validation croisée du DSS a confirmé ce comportement. Les récupérations pour les tâches de diagnostic ont proposé des solutions à modèle unique et multi-modèles en tenant compte des modèles des trois catégories. Les récupérations pour les prévisions proposaient également des solutions à un ou plusieurs modèles, mais les modèles proposés ne provenaient que de catégories physiques et axées sur les données. Cela permet de confirmer que l'espace de solution est bien couvert.

Pour chaque cas de test, les cas récupérés sont classés en fonction de la similitude des attributs du problème. Pour certains des tests, plusieurs cas récupérés avaient la même valeur de similarité maximale et/ou proposaient le même modèle pour réaliser une fonction PdM spécifique. Du point de vue de l'ingénierie des systèmes, cela représente deux limitations. La première fait référence à au moins deux cas avec la même similarité maximale car aucune information supplémentaire n'est fournie pour effectuer l'analyse de compromis pour sélectionner le composant le plus approprié. La seconde limitation est liée à la diversité des cas récupérés, en particulier lorsque deux cas ou plus suggèrent que le même modèle réalise une fonction spécifique. Un architecte en quête d'inspiration pour développer des solutions innovantes aura besoin que le DSS propose un ensemble diversifié de modèles. Ce sont des points d'amélioration à considérer à l'avenir pour le DSS. Une analyse plus détaillée peut également être effectuée qui inclut l'effort d'adapter la solution d'un cas récupéré au cas cible. Cela peut aider à effectuer une analyse de compromis sur les options rappelées et par conséquent à améliorer les performances du DSS.

D.7.2 Implementation d'un cas d'étude pratique

Dans le cadre de la validation, un exemple de mise en œuvre a été développé à partir des suggestions DSS dans l'étude de cas. L'étude de cas consiste en un ensemble de données de moteurs d'avion [Cha+21]. L'ensemble de données contient les enregistrements de dysfonctionnements de 128 moteurs d'avion dans des conditions de vol réelles et a été généré avec le modèle Commercial Modular Aero-Propulsion System Simulation (CMAPSS) développé par la NASA. Ces données sont appelées l'ensemble de données N-CMAPSS. Le but n'est pas seulement de tester les capacités du système CBR activé par l'ontologie (le DSS), mais aussi d'identifier les points d'amélioration pour le DSS. L'objectif de cette validation est de mettre en œuvre l'un des modèles proposés par le DSS pour remplir une fonction de maintenance prédictive pour la base de données N-CMAPSS. Cette implémentation permet de démontrer que DSS est capable de suggérer des composants appropriés pour les systèmes de maintenance prédictive, en complément de la validation croisée.

Afin de mieux valider les recommandations du DSS, l'un des modèles proposés a été pris en exemple et appliqué pour remplir la fonction de maintenance prédictive correspondante. Le DSS recommande la SOM et la régression logistique pour la modélisation de la santé comme les modèles les plus appropriés (similarité égale à 0,879) pour l'étude de cas N-CMAPSS. Construisant sur l'expérience de l'équipe de recherche en cartes d'auto-organisation (SOM), une implémentation du volet modélisation de la santé des moteurs est réalisée à l'aide d'un SOM. La section suivante fournit des explications supplémentaires sur l'implémentation de l'échantillon SOM et ses résultats préliminaires.

La méthodologie pour mettre en œuvre les cartes d'auto-organisation (SOM) pour la modélisation de la santé a été adoptée à partir de [Sch+20], (voir également l'annexe A). Les cartes auto-organisées sont des réseaux de neurones artificiels avec un entraînement non-supervisé qui sont capables de regrouper des instances de données en fonction des attributs d'instance. Un SOM est normalement constitué d'une couche carrée de neurones et les différents groupes après l'entraînement SOM peuvent être représentés graphiquement sur des cartes comme des régions bien définies. Il a été utilisé avec succès pour modéliser le processus de dégradation de différentes machines telles que les moteurs à réaction [MV18]. Dans ces cas, les neurones représentent la santé ou la dégradation de la machine dans un mode de fonctionnement spécifique. Le SOM entraîné aura une seule région, mais une transition du blanc (état optimal) au noir (état d'échec). Lors de l'évaluation de la santé ou de la dégradation d'une machine à l'aide du SOM entraîné, un neurone s'affichera sur la carte indiquant à quel point la dégradation est avancée ou à quel point la santé a diminué. C'est le véritable objectif de l'implémentation actuelle : obtenir un SOM entraîné capable de montrer une transition de l'état optimal à l'état défaillant. La Figure D.11 montre la carte auto-organisatrice entraînée avec le N-CMAPSS. Le résultat correspond au comportement attendu. Les différents modèles de dégradation ont été disposés sur la carte.

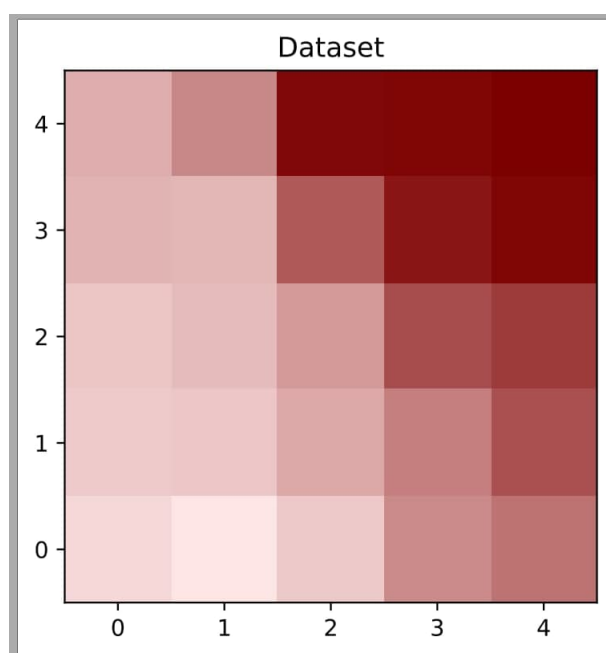


Figure. D.11: Carte auto-organisatrice formée avec l'étude de cas N-CMAPSS

Il est important de se rappeler que la validation d'un système d'aide à la décision (DSS) est une tâche difficile. La validation du SOM pour modéliser la santé du N-CMAPSS peut être utilisée pour valider indirectement le DSS ; cependant, il est important de souligner que d'autres implémentations seraient nécessaires pour confirmer les capacités et la précision du DSS proposé pour la sélection des modèles de maintenance prédictive. Dans l'implémentation actuelle, le N-CMPASS représente le cas cible pour lequel un nouveau système de maintenance prédictive doit être développé. Le système de recommandation CBR avec ontologie (également appelé DSS dans ce manuscrit) a proposé différents modèles pour remplir chaque fonction de maintenance prédictive. L'un des modèles proposés par le DSS pour modéliser la fonction de santé était la carte d'auto-organisation (SOM). La mise en œuvre du SOM a montré avec succès la tendance à la dégradation des moteurs à réaction, du fonctionnement nominal à l'état de défaillance en utilisant un sous-ensemble du N-CMPASS. Le SOM entraîné peut également être utilisé pour évaluer la santé/la dégradation d'un autre moteur du même type et dans les mêmes conditions de fonctionnement. Il est important de préciser que cette validation vise à démontrer la pertinence du modèle proposé par le DSS, mais davantage de comparaisons entre les modèles proposés sont nécessaires pour déterminer le meilleur modèle.

D.8 Conclusion et perspectives de travaux futurs

D.8.1 Résumé des contributions

Cette thèse comprend quatre articles qui ont été publiés ou acceptés pour publication dans des 2 revues scientifiques avec comité de relecture et 2 conférences internationales avec comité de relecture. Les principales contributions ont été regroupées dans les articles comme suit :

1. L'article de revue bibliographique résumait l'état de l'art en matière de diagnostics et de pronostics. Une proposition a été avancée pour différencier les modèles hybrides des approches multi-modèles.

Des tendances vers des approches multi-modèles ont été notées. Au moment de la rédaction de ce manuscrit, l'article de synthèse a été cité plus de vingt fois.

2. L'approche d'ingénierie des systèmes pour la conception de systèmes de maintenance prédictive propose, contrairement aux architectures génériques existantes, une approche systématique passant par une étape conceptuelle des systèmes de maintenance prédictive à partir des besoins et souhaits initiaux et ceci jusqu'à l'architecture logique. Cela permet d'aider l'architecte à déterminer les composants du système et à maintenir la traçabilité des besoins et souhaits initiaux des parties prenantes. Ces résultats ont été publiés dans une contribution de conférence à une conférence internationale.
3. Même si les ontologies n'étaient pas le domaine initial de ce projet de recherche, leur étude s'avérait nécessaire et ce travail de recherche a produit ses propres résultats. La création d'OMSSA, un modèle d'ontologie pour la sélection et l'évaluation des stratégies de maintenance, fournit un cadre terminologique qui peut être utilisé par des agents intelligents pour automatiser des tâches complexes de gestion de stratégies de maintenance. Une extension à OMSSA (OPMAD) est l'ontologie utilisée pour créer le « case base » et les mesures de similarité DSS pour la sélection des composants de maintenance prédictive. Le travail de recherche effectué dans la création de l'OMSSA a été consolidé dans un article de revue, actuellement accepté pour publication.
4. Un système d'aide à la décision (DSS) capable de consulter des cas réussis de mise en œuvre d'un système de maintenance prédictive peut aider un architecte système à sélectionner des modèles de maintenance prédictive appropriés pour effectuer des tâches de diagnostic et de prévision. Les premiers résultats de cette recherche ont été publiés dans une contribution de conférence à une conférence internationale.

D.8.2 Perspectives de travaux futurs

Pendant ce projet de recherche, il n'a pas été possible d'expliquer et de prouver tout. Il est donc important de mettre les choses en perspective pour repérer les axes d'amélioration et les lister comme perspectives de travaux futurs. Les perspectives de travaux futurs sont organisées en trois grands groupes :

1. Perspectives liées à l'approche d'ingénierie systèmes pour la conception de systèmes de maintenance prédictive
2. Perspectives pour OMSSA et OPMAD
3. Perspectives pour le DSS basé sur le raisonnement à base des cas

D.8.2.1 Perspectives liées à l'approche d'ingénierie systèmes pour la conception de systèmes de maintenance prédictive

- Affiner la liste des besoins, des souhaits et des exigences du système de maintenance prédictive : après la publication de l'article relatif à l'étape conceptuelle des systèmes de maintenance prédictive, plusieurs besoins, souhaits et exigences ont été identifiés qui n'avaient pas été initialement pris en compte. La liste des besoins, des souhaits et des exigences possibles pour les nouveaux systèmes de maintenance prédictive peut être améliorée. Ces listes peuvent être stockées dans des bases de données qui peuvent être utilisées par des agents intelligents automatisés pour aider les ingénieurs à évaluer les besoins et les souhaits initiaux d'un système de maintenance prédictive et à suggérer les exigences correspondantes des parties intéressées. Un DSS pourrait être mis au point pour définir les besoins des parties prenantes en fonction des besoins initiaux et des souhaits du nouveau système de maintenance prédictive.

- Inclure la technique phase des « trade-offs » dans le cadre systématique proposé : Les analyses de trade-off sont présentes à divers moments de la phase conceptuelle. L'intégration d'un outil d'analyse de trade-off (compromis) peut aider à améliorer le DSS et faciliter la sélection des composants de l'architecture.
- Inclure la formalisation des exigences système : un aspect important d'une approche d'ingénierie système consiste à obtenir les exigences système. Même lorsque la création de ces exigences est en théorie avant le processus d'architecture, en pratique, ces exigences sont généralement établies en parallèle avec le développement de l'architecture du système ou même à la fin, lorsque toutes les capacités des différents composants logiques sont connues. Ces exigences sont importantes pour la phase de conception détaillée. L'obtention de ces exigences sortait du cadre de la recherche actuelle, mais constitue un sujet complémentaire intéressant pour les recherches futures.

D.8.2.2 Perspectives pour OMSSA et OPMAD

- Affiner les classes et les relations : le développement d'ontologies est un sujet en constante évolution. Il existe plusieurs initiatives pour fournir des ontologies de niveau supérieur et intermédiaire dans différents domaines. Le domaine industriel ne fait pas exception. Des travaux supplémentaires seront nécessaires pour aligner OMSSA et OPMAD sur ces ontologies normalisées afin de favoriser leur intégration et leur réutilisation dans d'autres applications. Ce travail comprend l'affinement des classes et les relations entre elles.
- Extension du raisonnement en ontologie : OMSSA et OPMAD ont été utiles dans la recherche actuelle, mais en tant qu'ontologies, ils ont été sous-utilisés. Les ontologies ont diverses capacités qui n'ont pas été exploitées. L'une d'elles est la mise en œuvre de règles sémantiques. Cela élargit les options de raisonnement et peut être utilisé comme moyen complémentaire au système CBR.

D.8.2.3 Perspectives pour le DSS basé sur le raisonnement à base des cas

- Étendre l'utilisation de l'ontologie pour des fonctions de similarité plus locales : l'ontologie a été utile mais sous-utilisée. Certaines autres mesures de similarité qui ont été attribuées à une mesure de similarité de symboles binaires peuvent être réorganisées pour utiliser des similarités basées sur des ontologies.
- Utiliser l'ontologie pour estimer les poids pour le calcul de similarité globale : l'ontologie peut également être utilisée pour calculer les poids de chaque similarité locale lors du calcul de la similarité globale. Dans la première tentative de calcul de similitude globale effectuée dans cette enquête, toutes les similitudes locales ont reçu le même poids. Une amélioration du calcul de la similarité globale pourrait être de donner un rang d'importance à chaque similarité locale. Les poids peuvent être calculés à l'aide de l'ontologie peuplée.
- Étendre le système d'aide à la décision en développant d'autres phases du cycle de CBR, telles que la phase d'adaptation : la portée du DSS était dans la phase de reprise du cycle de CBR. Le système peut être étendu pour faciliter la phase d'adaptation des composants proposés pour le nouveau système de maintenance prédictive. Cette extension peut également inclure l'analyse de la compensation des différentes composantes suggérées par le DSS. Les résultats de la recherche actuelle ont montré plusieurs opportunités d'amélioration qui devraient être abordées à l'avenir avant la mise en œuvre éventuelle du DSS.
- Affiner le processus d'instanciation pour éviter les problèmes de diversité mais en gardant tout l'espace de solution couvert : l'instanciation du "case base" doit être affinée. Une opportunité d'amélioration

peut être de diviser l'espace de solution par les différentes fonctions de maintenance prédictive et de s'assurer que l'espace de solution est entièrement couvert par chacune d'entre elles. Des cas généralisés pour chaque fonction de maintenance prédictive peuvent résoudre le problème de diversité dans les cas de récupération.

- Analyse de compromis pour mieux sélectionner la technique parmi les propositions : comme pour le DSS, l'analyse des trade-offs peut aider l'architecte à discriminer entre les composants proposés par le DSS. Lors de la validation, il a été mentionné que parfois le DSS suggérait deux composants différents avec la même similitude. L'ajout d'attributs supplémentaires en tant qu'indicateurs de performance peut aider l'architecte à prendre la bonne décision pour son problème.

Summary in Spanish / Resumen en Español

Contenido

E.1	Introducción	224
E.1.1	Mantenimiento predictivo	224
E.1.2	Idea conceptual y arquitectura de un sistema	224
E.1.3	Declaración de investigación	225
E.1.4	Organización del resumen	226
E.2	Hacia un enfoque multimodelo en mantenimiento predictivo: una revisión literaria sistemática en diagnósticos y pronósticos.	227
E.3	De necesidades y deseos hasta una arquitectura lógica de sistemas de mantenimiento predictivo	229
E.4	Desarrollo de una ontología para el sistema de razonamiento basado en casos	231
E.4.1	Ontología para la selección y evaluación de estrategias de mantenimiento (OMSSA por sus siglas en inglés)	231
E.4.2	Ontología para la concepción de mantenimiento predictivo (OPMAD por sus siglas en inglés)	232
E.5	Desarrollo de sistemas de razonamiento basado en casos	233
E.5.1	Plataforma MyCBR	234
E.5.2	Representación de los casos	235
E.5.3	Desarrollo del motor de recuperación de componentes para sistemas de mantenimiento predictivo	236
E.6	Desarrollando un marco de trabajo para la selección de modelos de mantenimiento predictivo	236
E.7	Validación y discusión de resultados	238
E.7.1	Validación cruzada	238
E.7.2	Implementación del caso de estudio práctico	240
E.8	Conclusión y perspectivas de trabajo futuro	241
E.8.1	Resumen de contribuciones	241
E.8.2	Perspectivas de trabajo futuro	242

E.1 Introducción

E.1.1 Mantenimiento predictivo

Mantenimiento predictivo es una estrategia de mantenimiento que tiene como objetivo determinar el momento preciso para realizar las acciones de mantenimiento. No muy antes porque las acciones de mantenimiento podrían provocar el cambio de piezas que aún tienen una vida útil importante, lo que representa costos para las compañías; pero no muy tarde porque una falla inesperada puede ocurrir trayendo consigo una serie de consecuencias negativas. Mantenimiento predictivo es una alternativa a otras estrategias tradicionales de mantenimiento, como lo son mantenimiento correctivo y preventivo. En lugar de esperar a que una falla ocurra o realizar mantenimientos basados en intervalos fijos de operación, mantenimiento predictivo se basa en el diagnóstico del estado actual de los equipos y en el pronóstico de cuando una eventual falla podría ocurrir.

Mantenimiento predictivo está fuertemente relacionado con Mantenimiento Basado en Condición (CBM por sus siglas en inglés) y Administración de la Salud basado en Pronósticos (PHM por sus siglas en inglés). Tanto mantenimiento predictivo como estas dos disciplinas abarcan los diagnósticos y pronósticos para mantenimiento; en algunas ocasiones estos términos son incluso utilizados como sinónimos. Aparentemente, los términos de mantenimiento predictivo y CBM aparecieron en la década de 1940. Ambos términos se refieren a la estrategia de mantenimiento que pretende anticipar los fallos de las máquinas en función de su condición. Sin embargo, no es hasta principios de los años noventa cuando esta estrategia adquiere importancia gracias a la implantación de sistemas de monitorización y herramientas computacionales capaces de continuar con las tareas de diagnóstico. En cuanto al pronóstico, incluso cuando se menciona en el mantenimiento predictivo y CBM, siempre fue una disciplina inexacta. A principios de la década del 2000, la disciplina de PHM surgió con el objetivo de cubrir la brecha en investigación en pronósticos para mantenimiento predictivo. Desde entonces, la investigación en diagnóstico y pronóstico para el mantenimiento ha ganado mucha atención de la academia y la industria; gracias a los continuos incrementos en poder computacional y dados los beneficios de su implementación. En la última década, se han hecho diferentes contribuciones bajo los diferentes términos (mantenimiento predictivo, CBM y PHM) y se refieren al mismo campo de investigación.

E.1.2 Idea conceptual y arquitectura de un sistema

Según el manual de INCOSE [INC15], el ciclo de vida de un sistema puede dividirse en seis etapas genéricas (véase la Figura E.1). El ciclo de vida del sistema comienza con la etapa conceptual, cuyo objetivo es explorar posibles soluciones para satisfacer las necesidades iniciales de las partes interesadas. La fase conceptual va seguida de la fase de desarrollo, en la que se realiza un diseño detallado de los componentes del sistema y sus interfaces. Después del diseño detallado, comienza la etapa de producción que está dedicada a la implementación del sistema, lo que significa la fabricación, codificación, creación de los diferentes componentes y su integración final en el sistema. La verificación y validación del sistema forman parte de la fase de producción. Una vez validado el sistema, se entrega al cliente para su etapa de utilización. En la etapa de utilización, el sistema cumple la función para la que fue creado. La etapa de mantenimiento va en paralelo a la etapa de utilización durante el funcionamiento del sistema. Esta etapa de mantenimiento tiene como objetivo asegurar el estado de funcionamiento óptimo del sistema. Al final del ciclo de vida del sistema, la fase de eliminación tiene como objetivo gestionar adecuadamente los residuos del sistema cuando se retira de la operación.

El proceso de arquitectura del sistema tiene lugar dentro de la etapa conceptual. La etapa conceptual comienza por reunir todas las necesidades y deseos de las partes interesadas y formalizarlos en los requisitos

Concept stage	Development stage	Production stage	Utilization stage	Retirement
			Support stage	State

Figura. E.1: Etapas genéricas del ciclo de vida de un Sistema (en inglés) [INC15]

de las partes interesadas. Estos requisitos se utilizan luego para la creación de la arquitectura del sistema que servirá de base para el diseño detallado y la implementación del sistema. El desarrollo de la arquitectura se puede dividir en tres niveles [Roq18]; [INC15]: arquitectura funcional, arquitectura lógica y arquitectura física. La arquitectura funcional describe cómo las diferentes subfunciones de un sistema interactúan entre sí para cumplir un objetivo específico, pero no proporciona ningún detalle sobre los componentes que cumplen cada subfunción. La arquitectura lógica proporciona tanto detalle como sea posible de los componentes de la arquitectura y sus interfaces, pero no involucra a ninguna tecnología específica, lo que significa que la arquitectura lógica muestra componentes genéricos. La arquitectura física proporciona los detalles de las tecnologías que se asignarán para cada componente lógico. La unión de estos tres niveles de la arquitectura puede ser llamada como la arquitectura del sistema y puede entonces ser definida como los "conceptos fundamentales o propiedades de un sistema en su entorno encarnado en sus elementos, relaciones, y en los principios de su diseño y evolución" [ISO11]. En palabras simples, la arquitectura de un sistema muestra el cómo un conjunto de elementos, que podrían ser físicos o informativos, se organizan para cumplir un objetivo específico.

E.1.3 Declaración de investigación

El mantenimiento predictivo se lleva a cabo mediante sistemas especializados para realizar tareas de diagnóstico y pronóstico, para determinar el momento adecuado para desencadenar acciones de mantenimiento. El diseño y desarrollo de estos sistemas especializados sigue basándose en prueba y error. Existen varias arquitecturas genéricas en normas y estándares, como por ejemplo [MIM01], pero el diseño de un nuevo sistema de mantenimiento predictivo comienza mucho antes, en la etapa conceptual. Una arquitectura genérica no proporciona un vínculo con las necesidades y deseos iniciales de un nuevo sistema de mantenimiento predictivo y, a menudo, estas necesidades y deseos no se satisfacen con una arquitectura genérica. Además, una arquitectura genérica no proporciona ninguna guía para seleccionar las tecnologías adecuadas que puedan cumplir los componentes genéricos que propone [MV19].

El mantenimiento predictivo aborda tareas de diagnóstico y pronóstico, pero no todos los sistemas predictivos cubren las mismas funciones. Por ejemplo, un nuevo sistema de mantenimiento predictivo puede estar destinado a realizar diagnósticos en diferentes componentes de un sistema y se necesitarán varios módulos de diagnóstico del mismo tipo y no se incluirán módulos de pronóstico. Las arquitecturas genéricas no proporcionan un enfoque sistemático para diseños que sólo necesitan un subconjunto de los componentes genéricos propuestos o cuando se necesitan varios componentes del mismo tipo.

Existe un número importante de opciones para cumplir con las funciones de diagnóstico y pronóstico en un sistema de mantenimiento predictivo, y no hay directrices para ayudar al arquitecto a seleccionar los modelos, técnicas o algoritmos adecuados que pueden llevar a cabo estas funciones. La exploración del espacio de solución para determinar los componentes adecuados puede entonces ser compleja y de larga duración. Esta tesis tiene como objetivo facilitar el proceso de arquitectura de los sistemas de mantenimiento predictivo al permitir una manera más eficiente de explorar el espacio de solución y proponer los componentes más adecuados para la arquitectura del sistema. Antes de profundizar en el diseño de sistemas de mantenimiento predictivo, es importante entender el campo de investigación del mantenimiento predictivo en sí y las diferentes opciones disponibles para realizar diagnósticos y pronósticos. Se proponen

las siguientes preguntas de investigación para orientar el estudio del estado del arte en este tema:

1. ¿Cuáles son las tendencias actuales en diagnóstico y pronóstico en mantenimiento predictivo?
2. ¿Qué tipo de modelos, técnicas o métodos se utilizan para abordar el diagnóstico y el pronóstico en el mantenimiento predictivo?
3. ¿Cuáles son los principales desafíos que enfrenta el mantenimiento predictivo en el diagnóstico y el pronóstico?

Después del estudio de vanguardia, las preguntas de investigación se adaptan de acuerdo con los resultados del estudio y la motivación inicial de esta investigación relacionada con el diseño de sistemas de mantenimiento predictivo (ver sección E.2)

E.1.4 Organización del resumen

La sección E.2 aborda el estado del arte del mantenimiento predictivo basado en las preguntas iniciales de investigación introducidas en este capítulo. Esta sección está relacionada con el capítulo 2 de la tesis que se compone de un artículo publicado que incluye las tendencias actuales en modelos para diagnósticos y pronósticos en el campo del mantenimiento. La sección termina refinando las preguntas de investigación que motivaron el resto de la investigación.

La sección E.3 propone un enfoque de ingeniería de sistemas para el diseño de mantenimiento predictivo, específicamente en la etapa conceptual desde la reunión de las necesidades y deseos iniciales de las partes interesadas hasta la propuesta de una arquitectura lógica. Esta sección está relacionada con el capítulo 3 de la tesis que se compone de un artículo publicado en la conferencia y termina presentando la selección del componente de mantenimiento predictivo como el principal desafío que se abordará en los siguientes capítulos. Se propone un Sistema de Apoyo a las Decisiones (DSS por sus siglas en inglés) que combina ontologías y razonamiento basado en casos como posible solución para abordar la selección de componentes en el enfoque sistemático.

La sección E.4 explica uno de los elementos básicos del Sistema de Apoyo a las Decisiones: las ontologías. La investigación complementaria se ha realizado específicamente en ontologías que han resultado en un artículo de una revista, aceptado para su publicación en el momento en que este manuscrito fue escrito. El fondo teórico de las ontologías se presenta destacando su importancia en la comunidad de investigación debido a sus capacidades para modelar formalmente el vocabulario de un dominio en específico y realizar razonamiento con él. La sección E.4 explica el desarrollo de la ontología que más tarde se utiliza en el DSS para la selección de componentes. Esta sección está relacionada con el capítulo 4 de la tesis que incluye un artículo aceptado para publicación que trata de un modelo ontológico para la selección y evaluación de estrategias de mantenimiento (OMSSA por sus siglas en inglés).

La sección E.5 explica el segundo pilar del DSS: Razonamiento basado en casos (CBR). Los principios del paradigma CBR se introdujeron incluyendo las fases del ciclo CBR. En un primer intento, el DSS se centra en la fase de recuperación de CBR. Se proporciona una explicación de la implementación del motor de recuperación de CBR utilizando código de fuente abierta.

La sección E.6 explica el marco general de la integración de los componentes básicos del DSS y cómo encaja en el enfoque sistemático para diseñar sistemas de mantenimiento predictivo. Esta sección está relacionada con el capítulo 6 de la tesis que se compone de un artículo de conferencia publicado que es la continuación del artículo presentado en la sección E.2. La validación cruzada se realiza para demostrar las capacidades del DSS.

La sección E.7 tiene por objeto extender la validación del DSS. Se selecciona un caso de estudio para desarrollar el enfoque completo propuesto en la investigación actual y se implementa un ejemplo de modelo de mantenimiento predictivo para el caso de estudio basado en las sugerencias del DSS. Se explican y analizan los resultados de esta implementación.

En la sección E.8 se presentan las conclusiones de los trabajos realizados y se proponen perspectivas para una continuación de esas actividades de investigación en el futuro.

E.2 Hacia un enfoque multimodelo en mantenimiento predictivo: una revisión literaria sistemática en diagnósticos y pronósticos.

La revisión sistemática de la literatura muestra que el mantenimiento predictivo está ganando importancia en la comunidad académica, especialmente en los últimos 25 años. La Figura E.2 muestra el número de publicaciones que mencionan los términos "mantenimiento predictivo", "mantenimiento basado en condición" y "pronóstico y gestión de la salud" en los últimos 25 años en las fuentes de búsqueda consultadas.

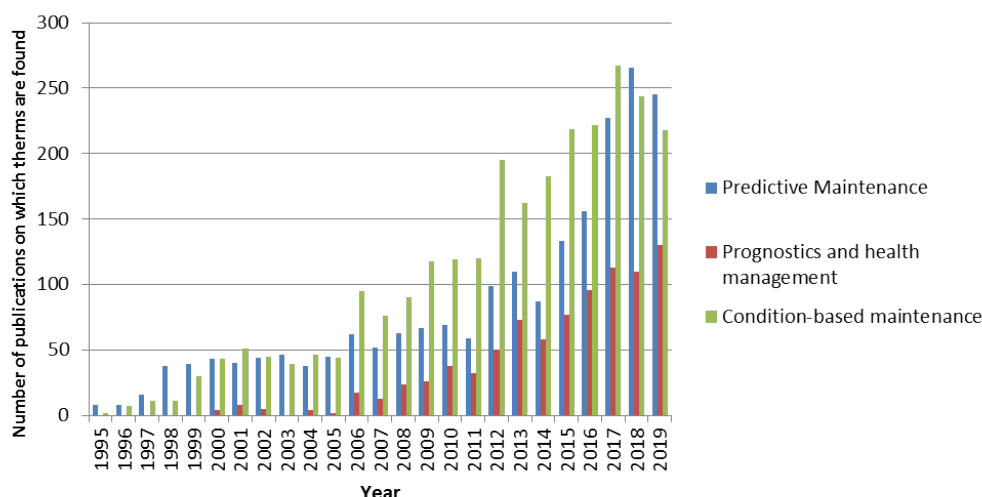
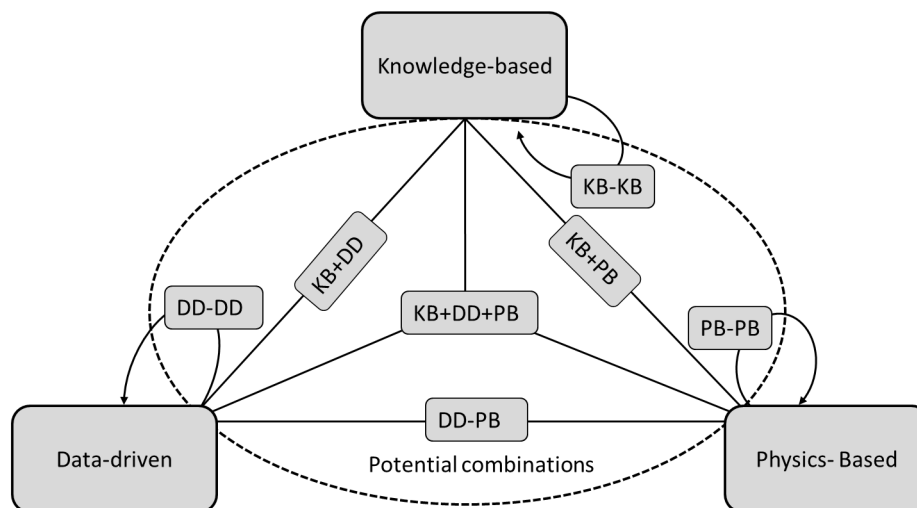


Figura. E.2: Número de publicaciones relacionadas con mantenimiento predictivo en los últimos 25 años.

La revisión bibliográfica se dividió en dos partes. En la primera se abordaron revisiones bibliográficas previas sobre el mantenimiento predictivo lo que ayudó a identificar los modelos utilizados para el mantenimiento predictivo y la evolución de las tendencias a lo largo de los años. Las taxonomías utilizadas para clasificar los diferentes modelos de diagnóstico y pronóstico en mantenimiento predictivo muestran ligeras variaciones en la terminología de un estudio a otro. Pueden extraerse dos enfoques principales: los enfoques de un solo modelo y los enfoques de varios modelos. Para los enfoques de un solo modelo se pueden identificar tres familias de modelos; para este estudio estas familias de modelos se denominarán de la siguiente manera: modelos basados en el conocimiento, modelos basados en datos y modelos basados en la física. Los enfoques multimodelo combinan al menos dos modelos de las tres familias de modelos mencionadas. Los enfoques multimodelo pueden tener diferentes configuraciones y a veces son nombrados como modelos híbridos.

La segunda parte de la revisión literaria se dedicó al estudio de las tendencias actuales en los diferentes enfoques de los modelos. La revisión literaria ha mostrado que existe una tendencia en la implementación de enfoques multimodelo ya que un solo modelo difícilmente satisface todas las funciones en un sistema de

mantenimiento predictivo. La Figura E.3 muestra todas las posibilidades de combinar modelos en enfoques multimodelo.



KB: Knowledge-based model. DD: Data-driven model. PB: Physics-based model.

Figura. E.3: Posibles combinaciones de modelos de mantenimiento predictivo.

La revisión bibliográfica permitió identificar los desafíos actuales para el mantenimiento predictivo:

- La extrapolación de soluciones existentes de mantenimiento predictivo en aplicaciones complejas, incluyendo múltiples componentes y sus fallas asociadas. La mayoría de las aplicaciones identificadas se centraron en un solo componente con un número limitado de fallos. Sin embargo, las aplicaciones de la vida real son frecuentemente sistemas complejos compuestos de muchos componentes y muchas fallas asociadas a cada componente y al propio sistema.
- La falta de un enfoque sistemático para diseñar y desarrollar sistemas de mantenimiento predictivo. Existen estándares, normas y arquitecturas genéricas para desarrollar nuevos sistemas de mantenimiento predictivo, como OSA-CBM. Sin embargo, sólo se centran en los componentes funcionales básicos del sistema y no abarcan aspectos importantes relativos a los indicadores de rendimiento o las limitaciones de contexto del sistema. Además, todavía no ofrecen una explicación consistente sobre qué modelos utilizar en función de las necesidades iniciales del sistema de mantenimiento predictivo. La falta de un enfoque sistemático limita la implementación de sistemas de mantenimiento predictivo en aplicaciones industriales a escala real.
- La fusión de diversas fuentes de datos de monitoreo de condición. Este desafío está relacionado con la extrapolación de los modelos actuales en mantenimiento predictivo a sistemas complejos. Los sistemas técnicos pueden tener diferentes tipos de fuentes de datos, por ejemplo mediciones de sensores, registros de mantenimiento, registros operacionales, documentos de diseño, etc. Se podrían reunir información importante de todas estas fuentes para implementar nuevos sistemas de mantenimiento predictivo.
- La incorporación de datos de influencia externa. El funcionamiento de los sistemas puede variar dependiendo de su contexto operativo. Los cambios en el contexto operativo pueden afectar directamente al rendimiento del sistema y, por consiguiente, a las lecturas del seguimiento sanitario. Puede activar

falsas alarmas sugiriendo la existencia de fallas, o puede impedir la identificación de fallas existentes. Esto podría abordarse mediante modelos complementarios capaces de incorporar la influencia externa con fines de mantenimiento predictivo.

- Manejo de la incertidumbre. La incertidumbre afecta directamente la precisión del diagnóstico y el pronóstico. Puede deberse a los datos recopilados o a las imperfecciones del modelo utilizado para el análisis. Puede afectar la fiabilidad de los resultados. La gestión de la incertidumbre es vital para los sistemas críticos sujetos a las regulaciones de las autoridades. Este es el caso de sistemas críticos como las centrales nucleares y las aeronaves en las que las regulaciones son restrictivas para mantener los estándares de seguridad y evitar eventos catastróficos.

El estudio del estado del arte motivó la mejora de las preguntas de investigación. El resto de la investigación fue impulsado por las siguientes preguntas:

1. ¿Cómo abordar la arquitectura y diseño de sistemas de mantenimiento predictivo?
2. ¿Cómo seleccionar un modelo adecuado o combinación de modelos dado un nuevo problema de mantenimiento predictivo para resolver?
3. ¿Cómo sugerir un enfoque adecuado para una solución de mantenimiento predictivo?
4. ¿Cómo puede un diseñador beneficiarse de la experiencia de los sistemas existentes para desarrollar nuevas soluciones de mantenimiento predictivo?

E.3 De necesidades y deseos hasta una arquitectura lógica de sistemas de mantenimiento predictivo

La primera pregunta de investigación obtenida al final del estudio del estado del arte es sobre el diseño de nuevos sistemas de mantenimiento predictivo. En la declaración de investigación presentada en la sección D.1 se explicó que el diseño y desarrollo de sistemas de mantenimiento predictivo todavía se basa en prueba y error. A pesar de la existencia de varias arquitecturas genéricas en normas y estándares, como por ejemplo [MIM01], la etapa conceptual de tales sistemas no está totalmente cubierta. Hay una brecha en el desarrollo de tales sistemas desde la recolección de necesidades y deseos para el nuevo sistema hasta la creación de la arquitectura que satisfaga las necesidades y deseos iniciales.

La creación de un nuevo sistema debe comenzar siempre escuchando a aquellos que están relacionados o interesados en el proyecto, conocidos como partes interesadas. Ellos proveen todas las necesidades y deseos para ser cumplidos por un nuevo sistema. Estas necesidades y deseos son la fuente de información para establecer la lista de requisitos de las partes interesadas del nuevo sistema. La investigación actual propone un enfoque de ingeniería de sistemas para cubrir la etapa conceptual de los sistemas de mantenimiento predictivo.

En el gráfico D.4 se muestran las diferentes etapas abordadas en el enfoque propuesto de ingeniería de sistemas. Comienza por reunir las necesidades y deseos iniciales de las partes interesadas. Estas necesidades y deseos se traducen en una lista formal de requisitos de las partes interesadas. Los requerimientos son priorizados y clasificados en requerimientos funcionales, de desempeño, estructurales y experienciales. Esta clasificación ayudará en la creación de la arquitectura del sistema. Los requisitos funcionales se utilizan para iniciar el proceso de arquitectura, específicamente para realizar el análisis funcional y crear el funcional. La arquitectura funcional se utiliza entonces para desarrollar la arquitectura lógica que sigue

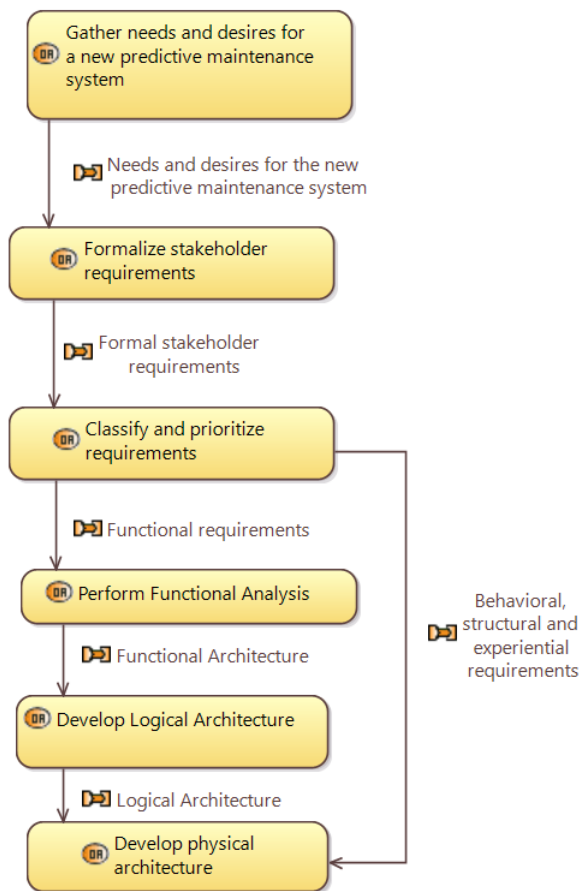


Figura. E.4: Pasos abordados en el enfoque de ingeniería de sistemas para el diseño de sistemas de mantenimiento predictivo

siendo genérica. Los requisitos de desempeño, estructurales y experienciales se utilizan para realizar la selección de componentes para cumplir con la arquitectura lógica y crear la arquitectura física.

Esta sección está relacionada con capítulo 3 de la tesis en el que se propuso un enfoque sistemático para abordar la etapa conceptual de los sistemas de mantenimiento predictivo a fin de dar respuesta a la primera pregunta de investigación perfeccionada. Los diferentes pasos desde la reunión de las necesidades y deseos iniciales de las partes interesadas hasta la definición de la arquitectura lógica se han cubierto y diferentes métodos se han propuesto para abordarlos. Sin embargo, el resto de las preguntas de investigación refinadas están aún por responder. En el mantenimiento predictivo, el espacio de solución es vasto y complejo como se muestra en el Capítulo 2 de la tesis. No hay reglas específicas que un arquitecto pueda seguir para seleccionar los modelos y enfoques adecuados para resolver nuevos problemas de mantenimiento predictivo. En esta investigación se propone una hipótesis para superar este problema: la implementación de un Sistema de Apoyo a la Decisión (DSS, por sus siglas en inglés) basado en el Razonamiento Basado en Casos (CBR, por sus siglas en inglés) y apoyado por ontologías podría ayudar al arquitecto a seleccionar componentes adecuados basados en experiencias pasadas extraídas de implementaciones exitosas de sistemas de mantenimiento predictivo.

E.4 Desarrollo de una ontología para el sistema de razonamiento basado en casos

El DSS se compone de dos bloques de construcción principales: Razonamiento basado en casos (CBR) y ontologías. CBR es un paradigma de razonamiento que busca resolver nuevos problemas basados en las experiencias de problemas similares resueltos en el pasado. CBR se aborda en el capítulo 5 de la tesis y en la sección D.5 del presente resumen, pero es necesaria una breve introducción para comprender el papel que desempeñan las ontologías en el DSS desarrollado en la investigación actual. Los sistemas CBR se desarrollan sobre un vocabulario base [Alt+12]; [Sán+12]. Este vocabulario es necesario para dar estructura a los casos almacenados de problemas resueltos en un caso basado, a las medidas de similitud que comparan el nuevo problema con los de la base de casos, y a los conocimientos necesarios para adaptar la solución recuperada para el nuevo problema (véase Figura E.5). Para efectos de esta tesis se elige una ontología para que sirva de marco terminológico (vocabulario base). Un modelo de ontología proporciona los términos, definiciones y relaciones entre los términos que se utilizan para construir la estructura de casos, las similitudes y el conocimiento de adaptación en el DSS. Este capítulo está dedicado a los antecedentes teóricos y el desarrollo del modelo de ontología para el DSS propuesto.

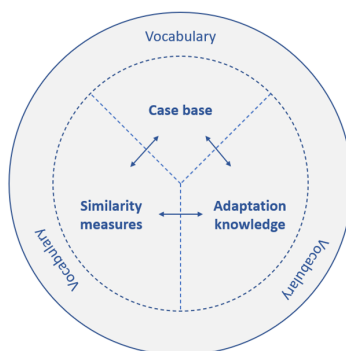


Figura. E.5: Razonamiento basado en casos desarrollado sobre un marco de vocabulario [Alt+12]

En ciencias de la información, una ontología es una descripción explícita formal de conceptos en un dominio del discurso, propiedades de cada concepto que describen sus características, atributos y restricciones [NM01]. Uno de los objetivos más comunes en el desarrollo de ontologías es "compartir un entendimiento común de la información de la estructura entre personas y agentes de software" [Gru93]; [Mus92]. Esto significa que el vocabulario utilizado por las personas en un dominio específico de conocimiento está habilitado para ser "legible por máquinas". Todos los conceptos en una ontología están representados por clases que están vinculadas por propiedades (también llamadas relaciones). Las ontologías se desarrollan con lenguajes formales. Uno de los más reconocidos es el Lenguaje de Ontología Web en su segunda versión (OWL2) que es apoyado por el World Wide Web Consortium (W3C) [Wor12].

E.4.1 Ontología para la selección y evaluación de estrategias de mantenimiento (OMSSA por sus siglas en inglés)

Una investigación complementaria al hilo principal de esta tesis permitió la creación de un modelo ontológico para la selección y evaluación de estrategias de mantenimiento (OMSSA por sus siglas en inglés). La creación de OMSSA sigue los mayores estándares y últimas tendencias en la creación de ontologías utilizando ontologías de alto nivel y medio nivel de referencia. Esto facilitará la reutilización e integración de OMSSA con otras ontologías relacionadas. La contribución de OMSSA fue consolidada en un artículo

científico que fue aceptado para su publicación en el momento que esta tesis fue escrita. OMSSA sirve de base para la creación del modelo ontológico que se usa para el sistema de ayuda a la decisión (DSS) propuesto en esta investigación.

E.4.2 Ontología para la concepción de mantenimiento predictivo (OPMAD por sus siglas en inglés)

Para esta sección se mantendrán los nombres de las clases de OPMAD en inglés para mantener congruencia con el modelo y los gráficos. OPMAD es una extensión de OMSSA y se han seguido los mismos estándares y metodologías para su desarrollo. La Figura E.6 presenta las clases y relaciones más importantes en OPMAD. Por limitación de espacio, las subclases no se muestran en la figura. Como el objetivo de OPMAD es apoyar el DSS para identificar modelos adecuados para los sistemas de mantenimiento predictivo, la explicación de las clases de ontología y sus relaciones comienza a partir de la clase Modelo de Mantenimiento Predictivo (*Predictive Maintenance Model*), y se basa en los términos presentados en la Figura E.6. Un modelo de mantenimiento predictivo se lleva en un módulo de mantenimiento predictivo (*Predictive Maintenance Module*) que es un componente de un sistema de mantenimiento predictivo (*Predictive Maintenance System*). Cada módulo de mantenimiento predictivo tiene una Función de Mantenimiento Predictivo (*Predictive Maintenance Function*), esta investigación se centra en funciones de diagnóstico y pronóstico.

El modelo de mantenimiento predictivo integrado en el módulo está directamente vinculado a un elemento mantenible (*Maintainable item*), que es la clase que describe el sistema mantenible para el que se desarrolla el sistema de mantenimiento predictivo. El elemento mantenible se clasifica por la clase tipo de elemento mantenible (*Maintainable item Type*) que se agregó con fines de cálculo de similitud en el DSS; este tipo de clases ayudan a comparar el nuevo problema a tratar con el problema resuelto en una base de casos de un sistema CBR. Los elementos mantenibles pertenecientes al mismo tipo comparten importantes características de degradación y por lo tanto se pueden proponer los mismos modelos de mantenimiento predictivo para resolver las mismas funciones de mantenimiento predictivo.

El elemento mantenible tiene su propia función (*Function*) que se ve afectada por una falla (*Failure*) que se manifiesta a través de un modo de falla (*Failure Mode*). Un elemento mantenible tiene uno o varios modos de fallo que también son objeto del modelo de mantenimiento predictivo. El modelo de mantenimiento predictivo tiene como entrada los datos de condición (*Condition*) que se utilizan para realizar diagnósticos y pronósticos. Dos cualidades importantes para la implementación de un modelo de mantenimiento predictivo es su tipo y configuración. El tipo de modelo de mantenimiento predictivo (*Predictive Maintenance Model Type*) clasifica el modelo dentro de las familias de modelos basados en conocimiento, basados en datos y basados en la física. Este tipo de clasificación ayuda a efectos de similitud y preferencias en el DSS. La configuración del modelo de mantenimiento predictivo (*Predictive Maintenance Model Configuration*) muestra si un modelo debe complementarse con otros modelos para cumplir su función; puede mejorar el rendimiento, pero aumenta la complejidad del desarrollo. El módulo de mantenimiento predictivo está calificado por la sincronización (*Module Synchronization*) y el indicador de rendimiento (*Module Performance Indicator*). Estas dos cualidades proporcionan información útil sobre cómo se deben probar y sincronizar los modelos de mantenimiento predictivos incorporados en los módulos con el elemento mantenible.

Toda esta información de los modelos de mantenimiento predictivo, elementos mantenibles, sus modelos de fallo, entre otras, se colecta en el caso de mantenimiento predictivo (*Predictive Maintenance Case*) que se documenta y publica a través de un artículo de mantenimiento predictivo (*Predictive Maintenance Article*), un tipo especial de artículo que se centra en el mantenimiento predictivo. A partir de estos artículos de mantenimiento predictivo también se incluyen algunos indicadores bibliométricos en la ontología que se utilizará en el sistema CBR. Estos indicadores bibliométricos se proporcionan como atributos de

solución para que el ingeniero pueda encontrar fácilmente la fuente de información de un caso específico. El título, el identificador y el año de publicación del artículo pueden utilizarse con fines de similitud y/o como fuente de información para más detalles de los modelos de mantenimiento predictivo y su implementación.

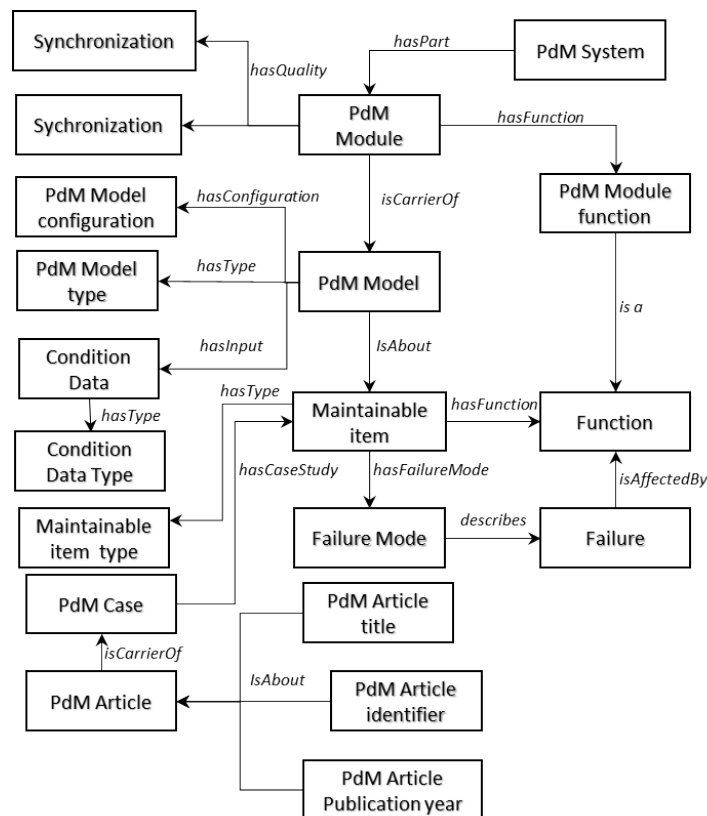


Figura. E.6: Clases and relaciones en OPMAD

E.5 Desarrollo de sistemas de razonamiento basado en casos

El razonamiento basado en casos (CBR) es una metodología de resolución de problemas basada en recuperación de soluciones previas para problemas similares [De +05]. El razonamiento basado en el caso se desarrolló bajo la filosofía de que los seres humanos piensan y razonan usando analogías y ejemplos, en lugar de estructuras SI-ENTONCES, estas últimas formando la base para el razonamiento basado en reglas. La solución del problema se realiza en un proceso cíclico de varios pasos: el ciclo CBR [AP94]. Este ciclo se compone de cuatro fases: recuperar, reutilizar, revisar y retener (ver Figura E.7).

El ciclo CBR se activa cuando se encuentra un nuevo problema. Esta primera fase tiene como objetivo recuperar los casos más similares de una base de conocimientos que almacena todos los casos anteriores. El caso objetivo (nuevo) se compara con los casos existentes en la base de conocimientos utilizando diferentes mediciones de similitud. El caso más cercano recuperado se propone como una posible solución en la fase de reutilización. Tal vez se necesite alguna adaptación para aplicar la solución en el caso previsto. Después de sugerir e implementar la solución se lleva a cabo la fase de revisión. Si la solución sugerida logra resolver el problema, se confirma y en la última fase, se retiene en la base de conocimiento para que pueda ser reutilizada en futuros problemas similares. En las subsecciones siguientes se proporcionan más detalles sobre cada fase de la RBC.

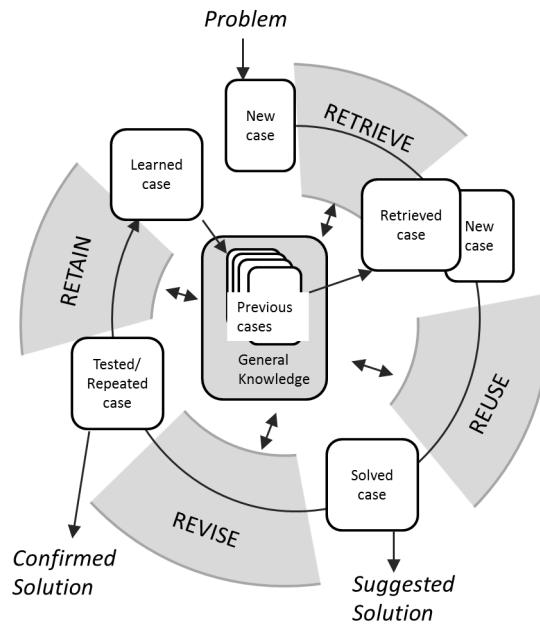


Figura. E.7: Ciclo de razonamiento basado en casos, ilustración inspirada en [AP94]

E.5.1 Plataforma MyCBR

El módulo CBR del DSS para la investigación actual se ha creado utilizando myCBR, una herramienta de búsqueda basada en similitudes de código abierto para el razonamiento basado en casos (CBR) [Alt+12]. Al desarrollar un sistema de recuperación de CBR usando myCBR el primer paso es determinar los atributos del caso. Los casos están representados por un vector acoplado de atributos:

Caso = [Atributos del problema, Atributos de la solución].

Los atributos de problema se utilizan para medir la similitud de un caso objetivo y los casos en una base de casos. Los atributos de la solución, como su nombre lo dice proveen información relevante relacionada con la solución del problema en cada caso almacenado en la base de casos.

Una vez los atributos han sido determinados, el segundo paso es la asignación de una medida de similitud local a cada atributo del problema. MyCBR proporciona varias opciones para calcular la similitud para cada atributo del problema. Dentro de las opciones disponibles, se han utilizado tres funciones de similitud para la investigación actual:

1. Similitud entero/flotante: esta función de similitud se utiliza para los atributos numéricos. La similitud se obtiene por una diferencia entre un valor de referencia y los valores de entrada de la función. Se necesita una función matemática para definir cómo disminuye la similitud a medida que los valores de entrada se alejan del valor de referencia. Esta función matemática puede ser lineal, exponencial o determinada por puntos discretos en el plano cartesiano.
2. Similitud de símbolo: esta medida de similitud es aconsejable para variables con un conjunto fijo de opciones. Estas opciones se organizan en una matriz de similitud y se dan algunos valores numéricos

para establecer la similitud entre las opciones. Esta función de similitud ha sido modificada para que algunos atributos de la investigación actual interactúen con el modelo ontológico. Los valores de la matriz de similitud se obtienen automáticamente a partir de las clases y relaciones de una ontología utilizando el enfoque de similitud basado en características propuesto por [Sán+12].

3. Similitud de cadena de caracteres: la similitud de atributos se obtiene con base en cadenas de texto abiertas. A diferencia de la similitud de símbolo que tiene un número limitado de opciones para definir la similitud, la similitud de cadena de caracteres solo tiene la restricción de tener oraciones o palabras como entrada. MyCBR ofrece tres opciones para calcular la similitud basada en cadenas: Igualdad, Ngram y Levenshtein. Para el estudio de caso actual, se seleccionó la función Levenshtein para calcular los atributos basados en cadenas de caracteres de similitud. La función Levenshtein ofrece un medio flexible para calcular la similitud basada en cada carácter de la cadena [Lev66]. Esto es especialmente útil cuando hay un amplio conjunto de opciones que son desconocidas al crear similitudes y también tolera pequeños errores ortográficos.

Después del cálculo de similitud para cada uno de los atributos del problema, se combinan estas similitudes individuales en una similitud global. Por medio de una función de agregación se calcula la similitud global en la que cada atributo tiene un peso específico. El objetivo de los pesos es dar especial importancia a los atributos de problemas. Para efectos de esta investigación, todos los atributos reciben el peso de 1, lo que significa que tienen la misma importancia. MyCBR ofrece dos opciones diferentes para calcular la similitud global: suma ponderada y distancia euclidiana.

- Suma ponderada: como dice el nombre, es la suma de similitudes considerando el peso de cada una.
- Distancia euclidiana: la distancia euclidiana entre dos puntos en el espacio euclidiano es un número, la longitud de un segmento de línea entre los dos puntos.

En esta tesis, ambas funciones de amalgamación pueden ser utilizadas por el usuario. Es importante señalar que para calcular la similitud global sólo se consideran los atributos disponibles. Esto significa que, si el arquitecto cuenta sólo con dos o tres atributos de los siete que describen el problema, la similitud global se calculará sobre la base de esos dos o tres atributos.

E.5.2 Representación de los casos

"Un caso es una pieza de conocimiento en un contexto particular que representa una experiencia que enseña una lección esencial para alcanzar la meta del razonador" [Kol93]. Los casos se representan a menudo en una forma acoplada [problema, solución], en la que se aplican similitudes en la parte del "problema" para que la parte de la "solución" se recupera. La representación del caso se compone de tres partes principales [BKP05]:

1. Definir los atributos del caso.
2. Definir la estructura del contenido del caso.
3. Organizar la base de casos.

La representación y estructura del caso se hace usando OPMAD. La Figura E.6 mostró los diferentes atributos (clases en ontología) del caso representado en OPMAD. Estas clases de ontología se pueden dividir en dos grupos diferentes: los atributos del problema y los atributos de la solución (mantienen sus nombres originales en inglés):

- **Atributos del problema**=[PdM Function,Maintainable Item,Maintainable Item Type, Condition Data Type,Module synchronization,PdM Article Publication year]
- **Atributos de l solución**=[PdM Model,PdM Model Configuration,PdM Model Type, Module Performance Indicator,PdM Article Identifier,PdM Article Title]

E.5.3 Desarrollo del motor de recuperación de componentes para sistemas de mantenimiento predictivo

La base de casos del DSS es una versión instanciada de OPMAD que ha sido poblada con casos exitosos de implementaciones de mantenimiento predictivo. El proceso para instanciar OPMAD a partir de una extensa revisión bibliográfica incluye la definición de las diferentes variables a buscar y las posibles opciones para cada variable; esto permite una mejor estructura de base de casos y facilita la recuperación. Para cada atributo de problema, se selecciona una similitud local entre las opciones posibles: entero, símbolo, basado en ontología, texto abierto. Tabla E.1 muestra las funciones de similitud asignadas a cada uno de los atributos del problema. Se han añadido dos funciones de agregación diferentes para calcular la similitud global entre un caso objetivo y los casos almacenados en la base de casos. Se ha desarrollado un GUI para facilitar la verificación y validación del motor de recuperación. El motor de recuperación desarrollado es el núcleo del DSS para la selección de componentes de mantenimiento predictivo. El siguiente capítulo está orientado a mostrar el marco completo del sistema CBR con ontología para la selección de componentes de mantenimiento predictivo. Los detalles de desarrollo del motor de recuperación se proporcionan en la guía de códigos del apéndice C.

Tabla E.1: Asignación de funciones de similitud

Attribute	Similarity function
PdM Function	Símbolo (Ontología)
Maintainable item	Cadena (Levenshtein)
Maintainable item type	Símbolo (igualdad)
Condition Data	Símbolo (Ontología)
Condition Data Type	Símbolo (igualdad)
Module synchronization	Símbolo (igualdad)
PdM Article Publication Year	Entero (función definida por puntos)

E.6 Desarrollando un marco de trabajo para la selección de modelos de mantenimiento predictivo

Un hito común en el desarrollo de la arquitectura de sistemas es la arquitectura lógica. En la arquitectura lógica se provee la mayor cantidad de detalle posible manteniendo los componentes genéricos. Estos componentes genéricos deben ser sustituidos por componentes específicos creando así la arquitectura física del sistema. Para la selección de los componentes se puede aplicar la creatividad estructurada. La creatividad estructurada en la arquitectura lógica se basa en analizar las posibles combinaciones de los diferentes componentes físicos/informativos que pueden ser seleccionados para satisfacer cada componente lógico. Esto permite la exploración del espacio de solución y puede ayudar al arquitecto a identificar soluciones innovadoras para un nuevo sistema. Si se identifican varias soluciones posibles, puede ser necesario un análisis de compensación para seleccionar la más adecuada. La arquitectura seleccionada sirve de base para el diseño detallado de los sistemas. Los conocimientos obtenidos del nuevo sistema implementado pueden ser utilizados por el arquitecto para desarrollar futuros sistemas. Existe una analogía entre el trabajo de

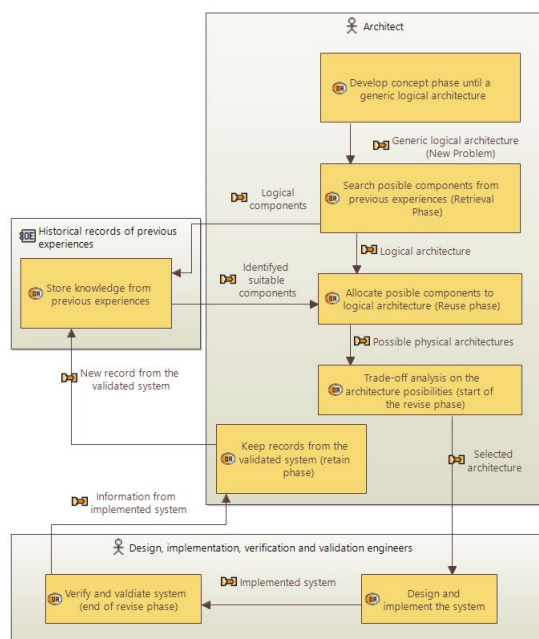


Figura. E.8: Analogía entre razonamiento basado en casos y las tareas que realiza un diseñador de sistemas, en notación Capella [Roq18]

creatividad estructurada realizado por el arquitecto para seleccionar componentes adecuados para cumplir con la arquitectura lógica y las cuatro fases del Razonamiento Basado en Casos: recuperar, reutilizar, revisar y retener. Una representación gráfica de esta analogía se presenta en Figura E.8 (en notación de Capella [Roq18]).

La analogía propuesta relaciona la fase de recuperación de CBR con la búsqueda realizada por el arquitecto en sistemas anteriores relacionados que pueden servir de inspiración para el desarrollo del nuevo sistema.

- La fase de reutilización de CBR sería la asignación de los componentes identificados para cumplir con la nueva arquitectura del sistema.
- La fase de revisión del CBR comenzaría por el análisis de compensación realizado por el arquitecto para identificar los componentes más adecuados. Esta fase continúa hasta la verificación y validación del sistema implementado que generalmente son realizados por otros actores distintos del arquitecto.
- Después de la validación del nuevo sistema, el arquitecto puede mantener los registros que se utilizarán en el futuro; esta sería la fase de retención de CBR.

Esta analogía sigue siendo genérica en cuanto al sistema que el arquitecto va a desarrollar. Todas las actividades son realizadas "manualmente" por el arquitecto y los registros de experiencias previas pueden no estar estructurados para facilitar su reutilización. Este enfoque manual puede funcionar cuando la cantidad de experiencias previas es limitada para que el espacio de solución a ser explorado por el arquitecto siga siendo manejable. Cuando el número de experiencias previas es alto o los requisitos complejos, la selección de los componentes más adecuados puede no ser fácil. Recuperar conocimiento de un número importante de arquitecturas anteriores puede consumir demasiado tiempo y se pueden perder opciones importantes.

Considerando esta analogía entre CBR y el trabajo de creatividad estructurada, se pueden proponer diferentes algoritmos para apoyar al arquitecto en las diferentes fases del trabajo de arquitectura. En un

primer paso, la investigación actual se centró en el desarrollo de un Sistema de Apoyo a la Decisión (DSS) capaz de realizar la búsqueda y recomendación de componentes físicos/informativos adecuados. Esto puede ayudar al arquitecto a ahorrar tiempo en la fase de concepto, y permite un análisis más amplio realizado por máquinas, análisis que difícilmente pueden ser abordados por los seres humanos.

La Figura E.9 presenta un concepto del DSS y cómo encaja en la analogía presentada en la Figura E.8. Este concepto se compone de tres partes principales: la base de casos, el motor de recuperación y la ontología específica del dominio. La base de casos almacena los casos del pasado en un formato estructurado para facilitar su reutilización. El arquitecto presentará al motor de recuperación la información del nuevo sistema en desarrollo y obtendrá un conjunto de los casos más similares recuperados de la base de casos que servirá de inspiración al seleccionar componentes adecuados para la arquitectura lógica.

Dentro de esta estructura la ontología juega un papel vital. Los casos, los atributos de estos casos y las similitudes entre las diferentes variables basadas en texto se describen a menudo en lenguaje natural. Modelar este lenguaje natural y hacerlo legible por máquina es necesario para automatizar la recuperación del caso. La integración de la ontología propuesta y el módulo de recuperación de CBR se explica con más detalle en una investigación complementaria presentada en el Apéndice B. La siguiente sección aborda la validación del DSS propuesto y la discusión de los resultados obtenidos.

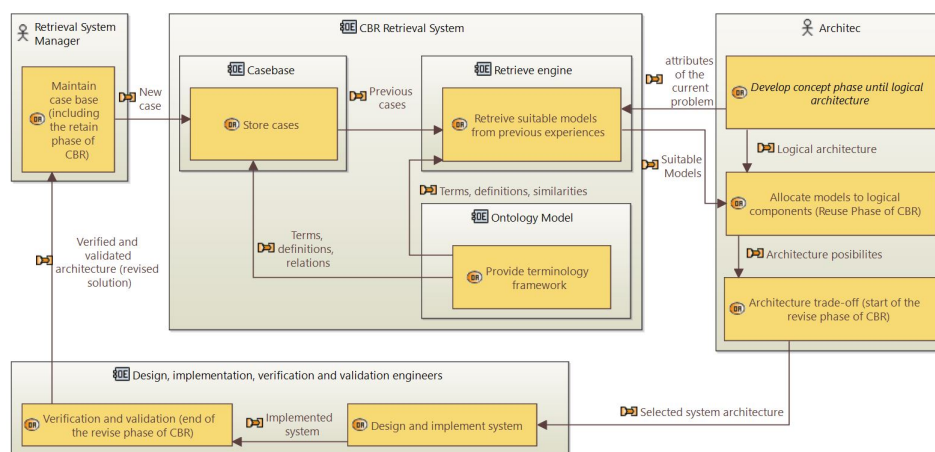


Figura. E.9: Idea conceptual de la incorporación del DSS para la selección de componentes de sistemas

E.7 Validación y discusión de resultados

La validación del DSS se divide en dos partes: una validación cruzada para confirmar la coherencia del DSS y una implementación de un modelo sugerido en un estudio de caso para probar las recomendaciones del DSS en un ejemplo práctico.

E.7.1 Validación cruzada

La validación cruzada es una técnica que se puede utilizar para probar la eficacia de los modelos de inteligencia artificial entrenados. Para el abordaje actual, se selecciona un enfoque de división de la prueba de tren para realizar la validación cruzada. Para el conjunto de pruebas, 63 de los 263 casos de la base de casos se extrajeron al azar y el resto se ha dejado en la base de casos como conjunto de entrenamiento. Para cada uno de los 63 casos extraídos, los atributos del problema fueron presentados al DSS usando GUI.

La atención se centró en los 10 casos más similares al realizar la recuperación. La Figura E.10 muestra un ejemplo de una prueba de recuperación en la que todos los atributos de problema del caso objetivo se corresponden con los atributos de solución correspondientes del caso 35. Esta coincidencia completa entre los atributos resulta en una similitud global de 1.

El espacio de solución de los modelos de mantenimiento predictivo se puede dividir en dos grupos como se explica en [Mon+20]. El primer grupo está compuesto por enfoques de modelo único, divididos en tres categorías principales: modelos basados en el conocimiento, modelos basados en datos y modelos basados en la física. El segundo grupo está compuesto por enfoques multimodelo, en los que al menos dos modelos de cualquiera de las tres categorías mencionadas se combinan para cumplir una función específica del sistema de mantenimiento predictivo. El mantenimiento predictivo está compuesto por tareas de diagnóstico y pronóstico. Según la revisión de la literatura [Mon+20], las tareas de diagnóstico como la detección de fallas, la identificación de fallas y la modelización del estado de salud pueden ser abordadas por modelos de las tres categorías de enfoques de modelo único o enfoques multimodelo. En cambio, para las tareas de pronóstico es difícil encontrar modelos basados en el conocimiento. Por ejemplo, las funciones de pronóstico, como la estimación de la vida útil restante y las funciones de previsión del estado siguiente, se cumplen normalmente mediante modelos basados en la física, modelos basados en datos y enfoques multimodelo que combinan modelos basados en la física y en datos. La validación cruzada del DSS permitió confirmar este comportamiento. Las recuperaciones para tareas de diagnóstico propusieron soluciones de modelo único y multimodelo considerando modelos de las tres categorías de modelos. Las recuperaciones para pronósticos también propusieron soluciones de modelo único y multimodelo, pero los modelos propuestos fueron sólo de categorías basadas en datos y en la física. Esto ayuda a confirmar que el espacio de solución está bien cubierto.

Input variables

PdM function : Remaining useful life estimation

Maintainable item type : Rotary machines

Maintainable item : Rolling bearings

Condition data type : Vibrations

Module synchronization : Off-line

Condition data : Time series

Variable weights

Remaining useful life estimation	1.0
Rotary machines	1.0
Rolling bearings	1.0
Vibrations	1.0
Off-line	1.0
Time series	1.0

Additional inputs

Number of cases to retrieve: 10

Aggregation function to use: euclidean

User dialog:

I found Case35 with a similarity of 1.000 as the best match. The 10 best cases shown in a table:

Case	Description
Case35 Sim = 1.000	Reference, Similarity and Input variables Reference: 35 Task: Remaining useful life estimation Case study type: Rotary machines Case study: Rolling bearings Online/Off-line: Off-line Input for the model: Time series Models: Convolutional Neural Network Input type: Vibrations Publication Year: 2018 Publication identifier: DOI: 10.1109/ACCESS.2018.2804930

SUBMIT QUERY

Figura. E.10: Ejemplo de recuperación de caso usando la interfaz del DSS

Para cada caso de prueba, los casos recuperados se clasifican en función de la similitud de los atributos del problema. Para algunas de las pruebas, el remolque o más casos recuperados tenían el mismo valor de similitud máximo y/o proponían el mismo modelo para cumplir una función PdM específica. Desde una perspectiva de Ingeniería de Sistemas, esto representa dos limitaciones. El primero se refiere a dos o más casos recuperados con la misma similitud máxima porque no se proporciona más información para realizar el análisis de compensación y se selecciona el más adecuado. La segunda limitación está relacionada con la diversidad en los casos recuperados, especialmente cuando dos o más casos recuperados sugieren

que el mismo modelo cumple una función específica. Un arquitecto que busca inspiración para desarrollar soluciones innovadoras necesitará del DSS para proponer un conjunto diverso de modelos. Estos son puntos de mejora para que el DSS sea considerado en el futuro. También se puede realizar un análisis más detallado que incluya el esfuerzo de adaptar la solución de un caso recuperado para el caso objetivo. Esto puede ayudar a realizar el análisis de compensación en las opciones recuperadas y, en consecuencia, mejorar el rendimiento del DSS.

E.7.2 Implementación del caso de estudio práctico

Como parte de la validación, se ha desarrollado un ejemplo de implementación a partir de las sugerencias del DSS en caso de estudio. El caso de estudio consiste en un conjunto de datos de motor de avión [Cha+21]. El conjunto de datos contiene los registros de fallos de funcionamiento de 128 motores de avión en condiciones reales de vuelo y se ha generado con el modelo Commercial Modular Aero-Propulsion System Simulation (CMAPSS) desarrollado por la NASA. Estos datos se llaman el conjunto de datos N-CMAPSS. El propósito no es sólo comprobar las capacidades del sistema CBR habilitado por ontología (el DSS) sino también identificar los puntos de mejora para el DSS. El objetivo de esta validación es implementar uno de los modelos propuestos por el DSS para cumplir una función de mantenimiento predictivo para la base de datos N-CMAPSS. Esta implementación ayuda a demostrar que el DSS es capaz de sugerir componentes adecuados para sistemas de mantenimiento predictivo, complementando la validación cruzada.

A fin de validar mejor las recomendaciones del Departamento de Seguridad, se ha tomado como ejemplo uno de los modelos propuestos y se ha aplicado para cumplir la función de mantenimiento predictivo correspondiente. Aprovechando la experiencia del equipo de investigación en Mapas de Auto Organización (SOM), se realiza una implementación del componente de modelado de salud utilizando el SOM. El DSS recomienda el SOM y la regresión logística para el modelado de salud como los modelos más adecuados (similitud igual a 0.879) para el estudio de caso N-CMAPSS. La siguiente sección proporciona explicaciones adicionales sobre la implementación del ejemplo SOM y sus resultados preliminares.

La metodología para implementar los Mapas de Auto-Organización (SOM) para la modelización de la salud ha sido adoptada de [Sch+20], (véase también el Apéndice A). Los mapas auto-organizados son redes neuronales artificiales con entrenamiento no supervisado que son capaces de agrupar instancias de datos dependiendo de los atributos de instancia. SOM se compone normalmente de una capa cuadrada de neuronas y los diferentes grupos después de la formación SOM se puede representar gráficamente en los mapas como regiones bien definidas. Se ha utilizado con éxito para modelar el proceso de degradación de diferentes máquinas como los motores a reacción [MV18]. En estos casos, las neuronas representan la salud o la degradación de la máquina en un modo operativo específico. El SOM entrenado tendrá una sola región, pero una transición de blanco (estado óptimo) a negro (estado fallido). Al evaluar la salud o la degradación de una máquina usando el SOM entrenado, una neurona saldrá en el mapa mostrando qué tan avanzada está la degradación o cuánto ha disminuido la salud. Este es el verdadero objetivo de la implementación actual: obtener un SOM capacitado capaz de mostrar una transición del estado óptimo al estado fallido. La figura E.11 muestra el mapa auto-organizativo entrenado con el N-CMAPSS. El resultado corresponde con el comportamiento esperado. Los diferentes modelos de degradación han sido ordenados en el mapa.

Es importante recordar que la validación de un Sistema de Apoyo a las Decisiones (DSS) es una tarea difícil. La validación del SOM para modelar la salud del N-CMAPSS puede utilizarse para validar indirectamente el DSS; sin embargo, es importante resaltar que se requerirían otras implementaciones que confirmen las capacidades y la precisión del DSS propuesto para la selección de modelos de mantenimiento predictivo. En la implementación actual, el N-CMAPSS representa el caso objetivo para el que se tiene que desarrollar un nuevo sistema de mantenimiento predictivo. El sistema de recomendación CBR con ontología (también llamado DSS en este manuscrito) propuso diferentes modelos para cumplir cada función

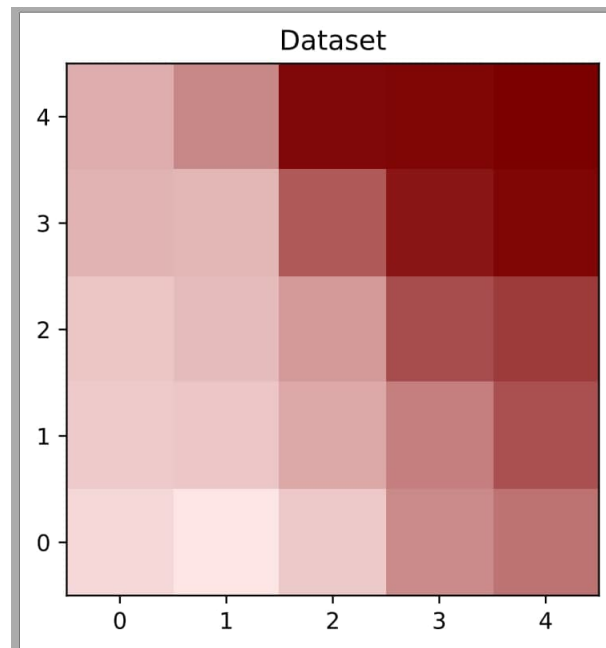


Figura. E.11: Mapa auto-organizativo entrenado con el caso de estudio N-CMAPSS

de mantenimiento predictivo. Uno de los modelos propuestos por el DSS para el modelado de la función de salud fue el Mapa de Auto-Organización (SOM). La implementación de SOM mostró con éxito la tendencia de degradación de los motores a reacción desde la operación nominal a la condición fallida utilizando un subconjunto del N-CMPASS. El SOM entrenado también se puede utilizar para evaluar la salud/ degradación de otro motor del mismo tipo y bajo las mismas condiciones de operación. Es importante aclarar que esta validación tiene por objeto demostrar la idoneidad del modelo propuesto por el DSS, pero se necesitan más comparaciones entre los modelos propuestos para determinar el mejor modelo.

E.8 Conclusión y perspectivas de trabajo futuro

E.8.1 Resumen de contribuciones

La tesis está compuesta por cuatro artículos que han sido publicados o aceptados para su publicación en revistas o conferencias internacionales. Las principales contribuciones de la investigación doctoral se han consolidado en los artículos de la siguiente manera:

1. El artículo de revisión bibliográfica resumió el estado del arte en diagnósticos y pronósticos. Se ha presentado una propuesta para diferenciar los modelos híbridos de los enfoques multimodelo. Se han señalado las tendencias hacia la aplicación de enfoques multimodelo. En el momento en que este manuscrito fue terminado, el artículo de revisión fue citado más de veinte veces. La revisión de la literatura puede ser vista como una contribución científica ya que proporciona un punto de partida para aquellos investigadores interesados en el mantenimiento predictivo.
2. El enfoque de ingeniería de sistemas para el diseño de sistemas de mantenimiento predictivo ha sido la primera contribución científica publicada en una conferencia internacional durante el desarrollo del doctorado. A diferencia de las arquitecturas genéricas existentes para el diseño de sistemas de

mantenimiento predictivo, el enfoque sistemático propuesto aborda la etapa conceptual de los sistemas de mantenimiento predictivo desde las necesidades y deseos iniciales hasta la arquitectura lógica. Ayuda al arquitecto a determinar con precisión los componentes del sistema y mantener la trazabilidad con las necesidades y deseos iniciales obtenidos de las partes interesadas.

3. Incluso cuando las ontologías no eran el ámbito principal de la investigación actual, una buena cantidad de trabajo de investigación se ha realizado en el dominio y ha producido sus propios resultados de investigación. La creación de OMSSA, un modelo de ontología para la selección y evaluación de estrategias de mantenimiento, proporciona un marco terminológico que puede ser utilizado por agentes inteligentes para automatizar las complejas tareas de gestión de estrategias de mantenimiento. Una extensión a OMSSA (OPMAD) es la ontología utilizada para crear la base de casos y las medidas de similitud del DSS para la selección de componentes de mantenimiento predictivo. El trabajo de investigación realizado en la creación de OMSSA se ha consolidado en un artículo de la revista que estaba bajo una segunda revisión después de una revisión importante en el momento en que este documento fue escrito.
4. El alcance principal de la investigación actual ha producido una contribución. Un sistema de apoyo a la toma de decisiones (DSS) capaz de consultar casos exitosos de implementación de sistemas de mantenimiento predictivo puede ayudar a un arquitecto de sistemas a seleccionar modelos de mantenimiento predictivo adecuados para realizar tareas de diagnóstico y pronóstico. Los primeros resultados de esta investigación se han publicado en un documento de conferencia que se ha publicado en una conferencia internacional.

E.8.2 Perspectivas de trabajo futuro

Mientras se trabaja en la investigación no es factible explicar y probar todo. Para lograr los objetivos de la investigación en un plazo fijo, algunos de los pasos intermedios de la investigación tienen que ser simplificados y a veces se toman decisiones pragmáticas. Al lograr los resultados, es importante poner las cosas en perspectiva para detectar los puntos de mejora y enumerarlos como perspectivas de trabajo futuro. La investigación nunca termina, y una de las partes principales de cualquier manuscrito de doctorado es indicar lo que faltaba y se espera que se haga con el fin de continuar la línea de investigación. Las perspectivas de la labor futura se organizan en tres grupos principales:

1. Perspectivas relacionadas con el enfoque de ingeniería de sistemas para el diseño de sistemas de mantenimiento predictivo.
2. Perspectivas relacionadas con las ontologías desarrolladas en la investigación actual.
3. Perspectiva relacionada con el sistema de recomendación basado en casos, habilitado para la ontología, para la selección predictiva de componentes.

E.8.2.1 Perspectivas relacionadas con el enfoque de ingeniería de sistemas para el diseño de sistemas de mantenimiento predictivo.

- Mejorar la lista de necesidades, deseos y requerimientos del sistema de mantenimiento predictivo: después de publicar el artículo relacionado con la etapa conceptual de los requerimientos que no fueron considerados originalmente. Se puede actualizar y refinar la lista de posibles necesidades, deseos y requisitos para nuevos sistemas de mantenimiento predictivo. Estas listas pueden almacenarse en bases de datos que pueden ser utilizadas por agentes inteligentes automatizados para ayudar a los ingenieros a evaluar las necesidades y deseos iniciales de un sistema de mantenimiento predictivo y

sugerir los requisitos correspondientes de las partes interesadas. Se podría desarrollar un DSS para definir los requisitos correctos de las partes interesadas basados en las necesidades iniciales y los deseos para el nuevo sistema de mantenimiento predictivo.

- Incluir la técnica de análisis de compensación en el marco sistemático: Los análisis de compensación están presentes en varios puntos durante la etapa conceptual. Por falta de tiempo, aún no se ha abordado. La integración de una herramienta de análisis de compensación puede ayudar a mejorar el DSS y facilitar la selección de los componentes de la arquitectura.
- Incluir la formalización de los requisitos del sistema: un aspecto importante de un enfoque de ingeniería de sistemas es la obtención de los requisitos del sistema. Incluso cuando la creación de estos requisitos es en teoría antes del proceso de arquitectura, en la práctica, estos requisitos se establecen normalmente paralelamente al desarrollo de la arquitectura del sistema o incluso al final, cuando se conocen todas las prestaciones de los diferentes componentes lógicos. Estos requisitos son importantes para la fase de diseño detallado. La obtención de estos requisitos estaba fuera del alcance de la investigación actual, pero es un tema complementario interesante para futuras investigaciones.

E.8.2.2 Perspectivas relacionadas con las ontologías desarrolladas en la presente investigación.

- Refinar clases y relaciones: el desarrollo de ontologías es un tema en rápida evolución. Hay varias iniciativas para proporcionar ontologías de nivel superior y medio en diferentes dominios. El dominio industrial no es la excepción. Será necesario seguir trabajando para alinear OMSSA y OPMAD a estas ontologías estandarizadas, para impulsar su integración y reutilización en otras aplicaciones industriales. Este trabajo incluye el refinamiento de las clases y las relaciones entre ellas.
- Extender el razonamiento en la ontología: OMSSA y OPMAD han sido útiles en la investigación actual, pero como ontologías, han sido infrautilizados. Las ontologías tienen varias capacidades que no fueron explotadas. Una de ellas es la implementación de reglas semánticas. Esto amplía las opciones de razonamiento y se puede utilizar como un medio complementario al sistema CBR.

E.8.2.3 Perspectiva relacionada con el sistema de recomendación basado en casos, habilitado para la ontología, para la selección predictiva de componentes.

- Ampliar el uso de la ontología para funciones de similitud más locales: como ya se ha mencionado, la ontología ha sido útil pero infrautilizada. Algunas otras medidas de similitud que han sido asignadas a una medida de similitud de símbolos binarios pueden ser reorganizadas para usar similitudes basadas en ontología.
- Utilizar la ontología para estimar los pesos para el cálculo de similitud global: relacionado con la observación anterior, la ontología también se puede utilizar para calcular los pesos de cada similitud local al calcular la similitud global. En el primer intento de cálculo de similitud global realizado en esta investigación, todas las similitudes locales recibieron el mismo peso. Una mejora en el cálculo de la similitud global podría ser dando un rango de importancia a cada similitud local. Los pesos se pueden calcular usando la ontología poblada.
- Ampliar el Sistema de Apoyo a las Decisiones desarrollando otras fases del ciclo CBR, como la fase de adaptación: El alcance del DSS estaba en la fase de recuperación del ciclo CBR. El sistema puede ampliarse para facilitar la fase de adaptación de los componentes propuestos para el nuevo sistema de mantenimiento predictivo. Esta ampliación también puede incluir el análisis de compensación de los diferentes componentes sugerido por el DSS. Los resultados de la investigación actual mostraron varias oportunidades de mejora que deberían abordarse en el futuro antes de la eventual implantación del DSS.

- Refinar el proceso de instanciación para evitar problemas de diversidad pero manteniendo todo el espacio de solución cubierto: Como se mencionó en el capítulo de validación, la instanciación de la base de casos debe ser refinada. Una oportunidad de mejora puede ser dividir el espacio de solución por las diferentes funciones de mantenimiento predictivo y asegurarse de que el espacio de solución está completamente cubierto para cada uno de ellos. Casos generalizados para cada función de mantenimiento predictivo pueden resolver el problema de la diversidad en los casos de recuperación.
- Análisis de compensación para seleccionar mejor la técnica entre las propuestas: ya se mencionó como la perspectiva de trabajo futuro para el enfoque sistemático para el diseño de sistemas de mantenimiento predictivo. En cuanto al DSS, el análisis de compensación puede ayudar al arquitecto a discriminar entre los componentes sugeridos por el DSS. En la validación se mencionó que a veces el DSS sugería dos componentes diferentes con la misma similitud. Agregar algunos atributos adicionales como indicadores de rendimiento puede ayudar al arquitecto a tomar la decisión correcta para su problema.

Résumé — La maintenance prédictive vise à déterminer le bon moment pour déclencher des actions de maintenance en fonction de l'état de santé d'un système. La maintenance prédictive est effectuée par des systèmes spécialisés dont la conception est encore basée sur des essais et des erreurs. Deux des principaux défis dans le développement des systèmes de maintenance prédictive sont l'absence d'une approche systématique pour aborder leur état conceptuel, et la sélection des composants appropriés pour traiter les tâches de diagnostic et de pronostic. Cette recherche vise à proposer des solutions possibles pour relever ces défis. Une approche d'ingénierie des systèmes est proposée pour aborder l'étape du concept, et un système d'aide à la décision (DSS) est proposé pour aider l'architecte à sélectionner les composants appropriés en fonction des mises en œuvre de maintenance prédictives réussies précédentes. Pour le développement du DSS deux technologies sont intégrées : Case-Based Reasoning et ontologies. La validation du DSS comprend la mise en œuvre de l'un des composants suggérés par le DSS pour accomplir une tâche de diagnostic dans une étude de cas de données de moteur à réaction simulée.

Mots clés : maintenance prédictive, architecture des systèmes

Abstract — Predictive maintenance aims at determining the right moment to trigger maintenance actions based on the health state of a system. Predictive maintenance is carried out by specialized systems whose design is still based on trial and error. Two of the main challenges in the development of predictive maintenance systems is the lack of a systematic approach to address their concept state, and the selection of suitable components to address the diagnostics and prognostics tasks. This research aims at proposing possible solutions to address these challenges. A systems engineering approach is proposed to address the concept stage, and a Decision Support System (DSS) is proposed to help the architect select suitable components based on previous successful predictive maintenance implementations. For the development of the DSS two technologies are integrated: Case-Based Reasoning and ontologies. The validation of the DSS includes the implementation of one of the suggested components by the DSS to fulfil a diagnostics tasks in a simulated jet-engine data case study.

Keywords: Predictive maintenance, systems architecture, knowledge reuse
